CAMPUSRESERVE

Intelligent Campus Resource Booking, Maintenance & Utilisation Analytics System

A Mini Project Report

Submitted in partial fulfilment of the requirements for

BCSE302P — DATABASE SYSTEMS LAB

Submitted by:

DURLAV DANI
Registration Number: 24BCE2361

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology, Vellore

Faculty: [INSERT FACULTY NAME]

Academic Year: 2026–2027
Fall Semester 2026–27

Date of Submission: [INSERT SUBMISSION DATE]

Individual Mini Project

Declaration of Originality

I, **Durlav Dani** (Registration Number **24BCE2361**), a student of the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, hereby declare that the mini project report entitled **"CampusReserve: Intelligent Campus Resource Booking, Maintenance & Utilisation Analytics System"**, submitted in partial fulfilment of the requirements of the course **BCSE302P — Database Systems Lab** during the Fall Semester of the academic year 2026–2027, is a record of an **individually developed academic project**.

I further declare that:

1. this project was carried out by me as an individual submission, and no team members or collaborators were involved;
2. the database design, SQL and PL/SQL implementation, application integration, testing and this report were prepared for this course submission;
3. every screenshot and every measured result presented as evidence in this report was produced from my own execution of the project;
4. all sources of information, books, papers, documentation and software tools used have been acknowledged in the References;
5. this work has not been submitted, in part or in full, for any other course, degree or diploma.

Tools disclosure: software development tools, including an AI-based coding assistant, were used during implementation and documentation. I have reviewed, executed and verified the work presented here.

Place: Vellore

Date: [INSERT SUBMISSION DATE]

_____

Durlav Dani (24BCE2361)

Abstract

Shared campus resources such as laboratories, seminar halls, classrooms and portable equipment are frequently double-booked because spreadsheets and e-mail cannot enforce booking rules or serialise competing requests. CampusReserve is a database-centred reservation, maintenance and utilisation-analytics system built on Oracle Database to address this problem. The relational schema of thirteen tables is normalised to Boyce–Codd Normal Form and protected by primary-key, foreign-key, unique, check and default constraints. All booking and maintenance writes pass through PL/SQL stored procedures that lock the resource row with `SELECT ... FOR UPDATE` before checking for overlapping intervals, so concurrent requests are serialised inside the database. Functions compute booking duration, availability and utilisation; two packages provide reservation workflow and analytics using explicit cursors, cursor `FOR` loops, REF cursors and a pipelined function; triggers enforce status transitions, detect overlaps and write an append-only audit trail. A React front end communicates with the database through a Node.js REST API. The scope covers registration, role-based approval, cancellation, maintenance, audit and analytics, together with reproducible experiments on conflict prevention, concurrency, rollback and execution plans before and after indexing.

Contents

Declaration of Originality **ii**

Abstract **iii**

1 Introduction **1**

1.1 Problem Statement 1

1.2 Objectives 1

1.3 Importance of a DBMS for this Problem 2

1.4 Scope 2

2 System Requirements **3**

2.1 Hardware Requirements 3

2.2 Software Requirements 3

2.3 Functional Requirements 3

2.4 Non-functional Requirements 4

3 Database Design **5**
3.1 System Architecture . . . 5
3.2 Entities, Attributes and Relationships . . . 5
3.3 Mapping the ER Model to Relations . . . 8
3.4 Schema Definitions with Constraints . . . 8
3.5 Normalisation . . . 9
3.5.1 Unnormalised form . . . 10
3.5.2 First normal form . . . 10
3.5.3 Functional dependencies . . . 11
3.5.4 Second normal form . . . 11
3.5.5 Third normal form . . . 11
3.5.6 Boyce–Codd normal form . . . 12
3.6 Indexes . . . 12

4 Implementation **13**
4.1 SQL Coverage of the Syllabus . . . 13
4.2 Functions . . . 14
4.3 Stored Procedures and Transaction-safe Booking . . . 16
4.3.1 Why a simple overlap check is not enough . . . 16
4.3.2 Concurrency-safe design . . . 16
4.4 Packages . . . 18
4.5 Triggers . . . 20
4.6 Cursors . . . 21

5 Front-End and Database Integration **23**
5.1 Integration Architecture . . . 23
5.2 Interface Design . . . 23
5.3 Screenshots . . . 24

6 Verification and Testing **26**
6.1 Test Strategy . . . 26
6.2 Execution Status . . . 26
6.3 Test Cases . . . 26
6.4 Experiment E2: Simultaneous Booking Attempts . . . 28
6.5 Transaction Rollback . . . 29
6.6 Experiment E3: Execution Plans Before and After Indexing . . . 30

7 Results and Discussion **32**
7.1 Achievements . . . 32
7.2 Discussion of the Research Question . . . 32

7.3 Challenges Faced . . . 33
7.4 Limitations and Possible Improvements . . . 33

8 Conclusion **34**
8.1 Learning Outcomes . . . 34
8.2 Future Scope . . . 34

References **35**

A Installation and Execution **37**

B Application Error Codes **38**

C Additional Diagrams and Evidence **39**

D Source Code **41**
D.1 Stored procedures (06_procedures.sql) . . . 41
D.2 Triggers (09_triggers.sql) . . . 48

List of Figures

3.1 System architecture: React front end, Express API and Oracle database objects 5
3.2 Entity–Relationship diagram of CampusReserve (crow's-foot notation) . . . 7
3.3 Relational schema: primary keys and foreign-key references between the thirteen tables . . . 8
3.4 Normalisation steps from the unnormalised booking register to 3NF/BCNF . . 10

4.1 Function outputs from verification script section V8 (own execution) . . . 16
4.2 Two sessions competing for the same slot: the row lock serialises them . . . 17
4.3 REF cursor output of pkg_resource_analytics.dashboard_summary and pipelined day_slots (own execution) . . . 19
4.4 Cursor FOR loop utilisation report printed with DBMS_OUTPUT (own execution) . . . 22

5.1 Overview dashboard with database-backed KPI tiles and booking trend . . . 24
5.2 Availability calendar fed by the resource_schedule REF cursor . . . 24
5.3 New booking form: live availability check and pipelined hourly slots . . . 24
5.4 Overlap rejected by sp_create_booking and shown to the user (HTTP 409) . . . 24
5.5 Approval queue for a coordinator . . . 25

5.6 Maintenance board . . . 25
5.7 Analytics computed by PKG_RESOURCE_ANALYTICS . . . 25
5.8 Booking detail with trigger-written change history . . . 25
5.9 Responsive layout at 390 px width . . . 25

6.1 Output of 10_test_cases.sql: PASS/FAIL lines (part 1) . . . 28
6.2 Output of 10_test_cases.sql: remaining cases and totals . . . 28
6.3 API unit tests (npm test) in the terminal . . . 28
6.4 Output of concurrency-experiment.js (naive versus locked) . . . 29
6.5 Two SQL sessions: session B blocks until A commits, then receives ORA-20005 29
6.6 Transaction tests A1–A3 . . . 30
6.7 DBMS_XPLAN output for Q1 before indexing . . . 30
6.8 DBMS_XPLAN output for Q1 after indexing . . . 31
6.9 Timing harness output and plan cost summary . . . 31

C.1 Booking life cycle enforced by sp_create_booking and trg_bookings_biu . . . 39
C.2 run_all.sql completing with “All objects VALID” . . . 39
C.3 Verification V1–V2: every object VALID, no compilation errors . . . 39
C.4 Verification V4–V5: constraints per table; no unindexed foreign keys . . . 40
C.5 Two-session naive check-then-insert and the resulting overlap query . . . 40

List of Tables

2.1 Hardware requirements . . . 3
2.2 Software, languages and tools . . . 3

3.1 Relations and their purpose . . . 6
3.2 Constraint inventory . . . 9
3.3 Functional dependencies identified in BOOKING_1NF . . . 11
3.4 Access-path indexes . . . 12

4.1 Database objects implemented . . . 13
4.2 Syllabus experiments and where they are demonstrated . . . 14
4.3 Functions: parameters and results . . . 15
4.4 Stored procedures . . . 16
4.5 Triggers . . . 20

5.1 Screens and the database objects behind them 23

6.1 Execution status of each test suite at the time of writing 26
6.2 Functional test cases 26
6.3 Concurrency experiment results (record every run) 29
6.4 Execution-plan experiment (fill in from your own output) 30

B.1 Application error codes raised by PL/SQL and their HTTP mapping 38

1. Introduction

1.1 Problem Statement

A university shares a large number of expensive facilities between schools, clubs and research groups. In most departments these are reserved through e-mail requests, paper registers or shared spreadsheets. Such tools have four structural weaknesses:

1. **No conflict prevention.** Nothing stops two people from reserving the same room for overlapping times, and the clash is discovered only when both groups arrive.
2. **No rule enforcement.** Capacity limits, operating hours, minimum notice and approval requirements depend on people remembering them.
3. **No reliable history.** Spreadsheet cells can be overwritten without trace, so it is impossible to establish who approved or cancelled a booking.
4. **No usage evidence.** Administrators cannot tell which rooms are over-subscribed and which are idle, so maintenance and investment decisions are guesswork.

The first weakness is the hardest. An application that simply *reads* the bookings for a time slot and then *inserts* a new booking if it sees none is incorrect under concurrency: two requests that arrive at the same moment both see an empty slot and both succeed. Correctness therefore has to be guaranteed where the data lives, in the database.

1.2 Objectives

The objectives of this individual project are to:

1. design a normalised relational schema (1NF to BCNF) for users, roles, departments, buildings, resources, bookings, approvals, maintenance, blackouts, notifications, audit logs and utilisation snapshots;
2. enforce data integrity declaratively through primary-key, foreign-key, unique, check, not-null and default constraints;
3. implement every booking and maintenance write in PL/SQL stored procedures that are transaction-safe and concurrency-safe;
4. demonstrate functions, procedures, packages, triggers and cursors that are genuinely used by the application;
5. build a web front end and REST API that use the database objects instead of duplicating their logic;

6. verify the system with test cases, and measure the effect of indexing using execution plans; and
7. answer the research question: *How can relational database integrity constraints, row-level locking and indexing improve correctness and performance in shared-resource reservation systems?*

1.3 Importance of a DBMS for this Problem

A database management system offers exactly the guarantees that the spreadsheet approach lacks [1, 2]. The relational model separates facts into relations linked by keys, which prevents update anomalies [4]. Constraints are checked by the DBMS for every client, so invalid data cannot enter through a forgotten code path. Transactions provide atomicity, consistency, isolation and durability: a booking and its notification are written together or not at all. Row-level locking allows many users to work concurrently while serialising only the operations that compete for the same resource [9]. Triggers make auditing automatic. Indexes and the cost-based optimiser make the same queries fast as the data grows. Finally, aggregate queries and views turn operational data into utilisation analytics without a separate system.

1.4 Scope

In scope: registration and sign-in, four roles (student, faculty, coordinator, administrator), resource discovery, a week calendar, booking creation, approval, rejection and cancellation, maintenance tickets that can take a resource out of service, blackout periods, notifications, an audit trail, analytics dashboards and an administrative interface. Out of scope: payment, recurring bookings, integration with the university's timetable system, and any artificial-intelligence or machine-learning component; the word "intelligent" in the title refers only to rule-based scheduling checks and database analytics. All people and bookings in the database are synthetic demonstration data.

2. System Requirements

2.1 Hardware Requirements

Table 2.1: Hardware requirements

Component	Minimum for development and demonstration
Processor	64-bit x86-64 or ARM64, 2 cores (4 recommended)
Memory	8 GB RAM (Oracle Database Free uses up to 2 GB)
Storage	15 GB free (database image, Node.js dependencies, LaTeX)
Network	Internet access for package installation and optional cloud deployment

2.2 Software Requirements

Table 2.2: Software, languages and tools

Layer	Choice
DBMS	Oracle Database 19c or later; developed for Oracle Database 23ai Free (local) and Oracle Autonomous Database (cloud). SQL `BOOLEAN` and `DROP ... IF EXISTS` are avoided for 19c compatibility.
Database languages	SQL, PL/SQL
Database tools	SQL Developer or SQLcl; `DBMS_XPLAN`, `DBMS_STATS`, `DBMS_OUTPUT`
Back end	Node.js 20+ (built with 22), Express 5, node-oracledb 6+ in Thin mode (no Oracle Client needed), bcryptjs, jsonwebtoken, helmet, express-rate-limit, zod
Front end	React 19, TypeScript, Vite, Tailwind CSS 4, React Router, Recharts, Lucide icons
Testing	Node.js test runner, Supertest, SQL test scripts
Documentation	XeLaTeX (this report), Mermaid (diagrams), Markdown
Deployment	GitHub, Netlify (front end), Render (API), Oracle Autonomous Database
Operating system	Windows 10/11, macOS or Linux

2.3 Functional Requirements

FR1. Users can register, sign in and update their profile; new accounts are students.

FR2. Users can search resources by text, category, building and capacity.

FR3. Users can see a week calendar and hourly availability for any resource.

FR4. Users can request a booking; the system accepts it only if the interval is valid, within operating hours, within the category's maximum duration and notice period, within capacity, outside blackouts, and free of overlapping live bookings.

FR5. Coordinators can approve or reject requests for resources managed by their department; administrators can decide any request.

FR6. Requesters can cancel their own live bookings; approvers can cancel any.

FR7. Any user can report a fault; staff can take a resource out of service; coordinators and administrators move tickets through their life cycle.

FR8. Every booking, maintenance and resource-status change is recorded in an audit log that cannot be modified.

FR9. Staff can view utilisation, trends, department usage and peak hours computed from the database.

FR10. Administrators can manage resources, blackout periods and user roles.

2.4 Non-functional Requirements

Correctness under concurrency; security (hashed passwords, token authentication, role checks, bind variables, no secrets in the client); responsiveness on mobile, tablet and desktop; accessibility (keyboard navigation, focus indicators, contrast); and reproducibility (all scripts re-runnable, all measurements repeatable).

3. Database Design

3.1 System Architecture

Figure 3.1 shows the three tiers. The browser never talks to the database directly. The API authenticates the user, validates input, opens one transaction per request, sets the Oracle session `CLIENT_IDENTIFIER` to the user's ID for auditing, and calls stored procedures, package functions and views. Business rules live in the database so that they hold for every client, including SQL*Plus sessions.

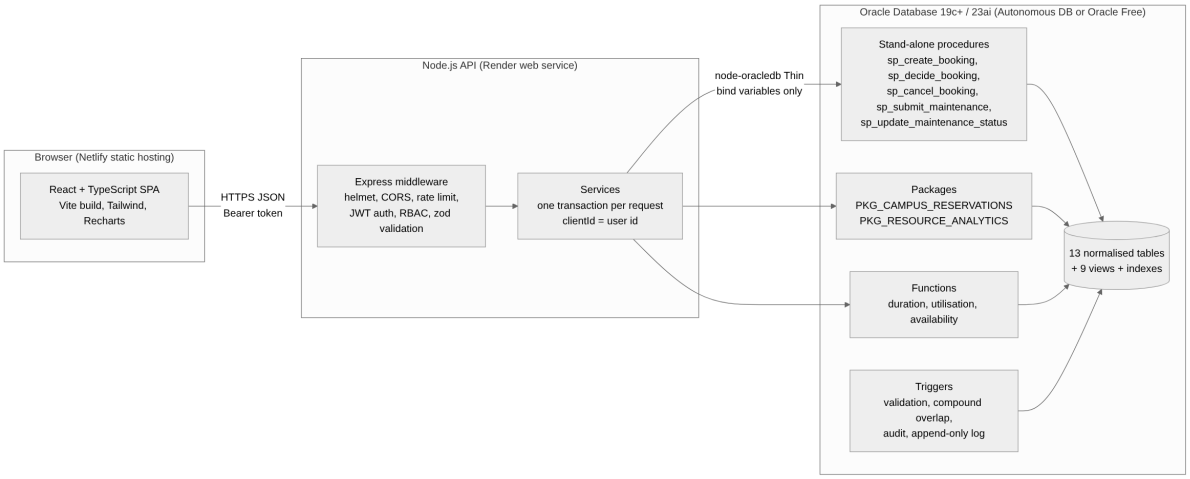


Figure 3.1: System architecture: React front end, Express API and Oracle database objects

3.2 Entities, Attributes and Relationships

Table 3.1 lists the thirteen relations and the reason each exists. The Entity–Relationship diagram in Figure 3.2 shows attributes, keys and cardinalities.

Table 3.1: Relations and their purpose

Relation	Purpose
ROLES	authorisation level; flags `can_approve`, `can_administer`
DEPARTMENTS	schools and offices that own users and resources
USERS	everyone who can sign in (bcrypt password hash, unique e-mail)
BUILDINGS	physical buildings
RESOURCE_CATEGORIES	booking policy shared by a class of resources (approval, maximum hours, notice)
RESOURCES	anything bookable: status, capacity, operating hours, JSON features
BOOKINGS	one reservation of one resource for one half-open interval $[s, e)$
BOOKING_APPROVALS	the single decision on a booking (one-to-one with BOOKINGS)
MAINTENANCE_REQUESTS	fault tickets; `blocks_booking` takes a resource out of service
RESOURCE_BLACKOUTS	scheduled unavailability (exams, holidays, servicing)
AUDIT_LOGS	append-only change history written only by triggers
NOTIFICATIONS	in-application messages
UTILISATION_SNAPSHOTS	dated output of the utilisation reporting cursor

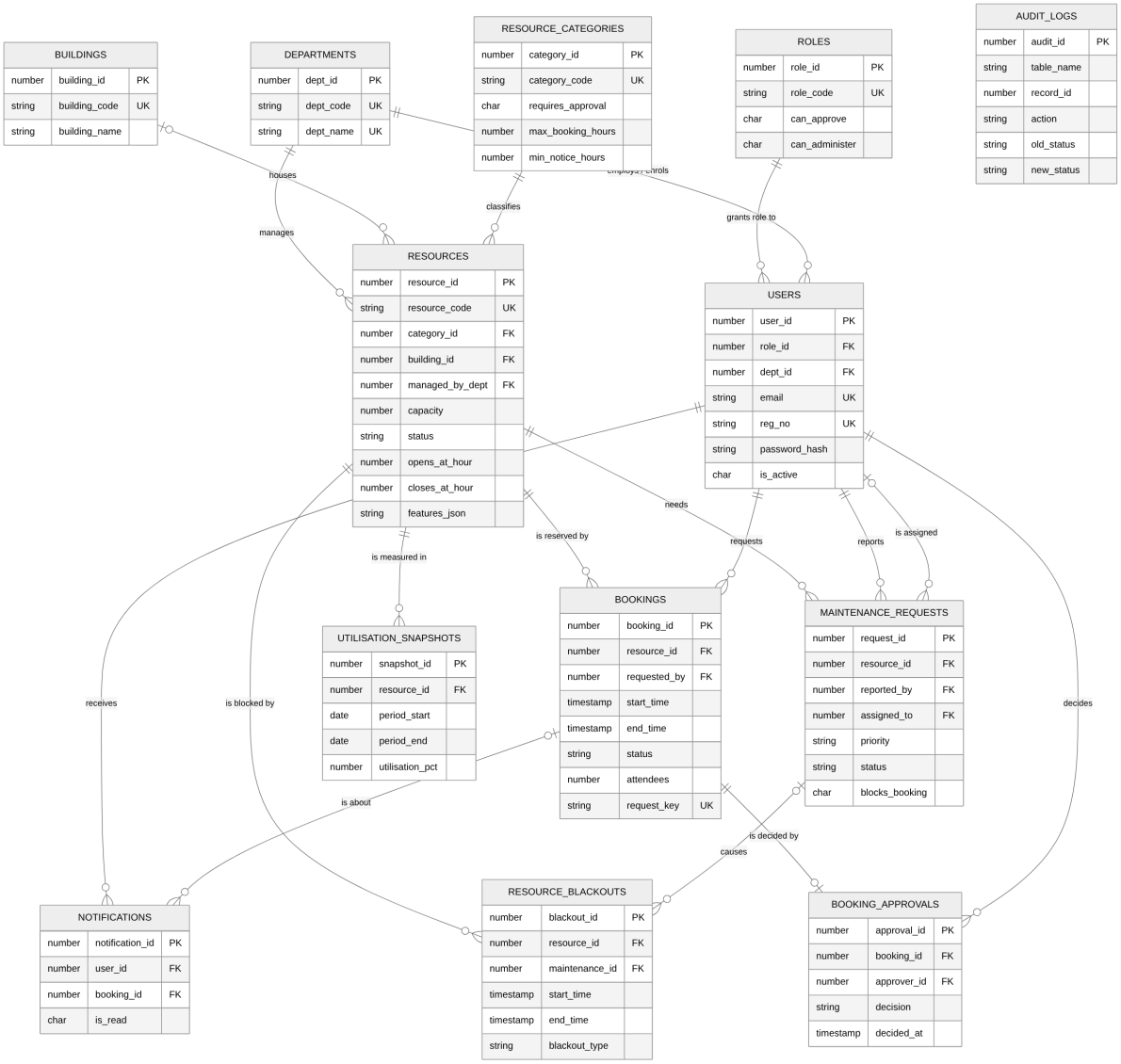


Figure 3.2: Entity–Relationship diagram of CampusReserve (crow's-foot notation)

Key relationship decisions:

- **BUILDINGS 0..1 to many RESOURCES.** Portable equipment has no building, so `building_id` is nullable.
- **BOOKINGS 1 to 0..1 BOOKING_APPROVALS.** A unique constraint on `booking_id` guarantees at most one decision; re-deciding is impossible at the data level.
- **USERS to MAINTENANCE_REQUESTS** appears twice, as reporter (mandatory) and assignee (optional).
- **AUDIT_LOGS has no foreign key.** Audit rows must outlive the rows they describe and must never block a delete; they reference records through `(table_name, record_id)`.

3.3 Mapping the ER Model to Relations

Each strong entity became a table with a surrogate identity primary key; natural codes (e-mail, resource code, department code) are kept as `UNIQUE` candidate keys. Each one-to-many relationship became a foreign key on the "many" side. The one-to-one decision relationship became a foreign key with a unique constraint. Multi-valued descriptive features of a resource (projector, GPU count) are stored in a validated JSON column because they are never joined or aggregated. Figure 3.3 shows the resulting relational schema.

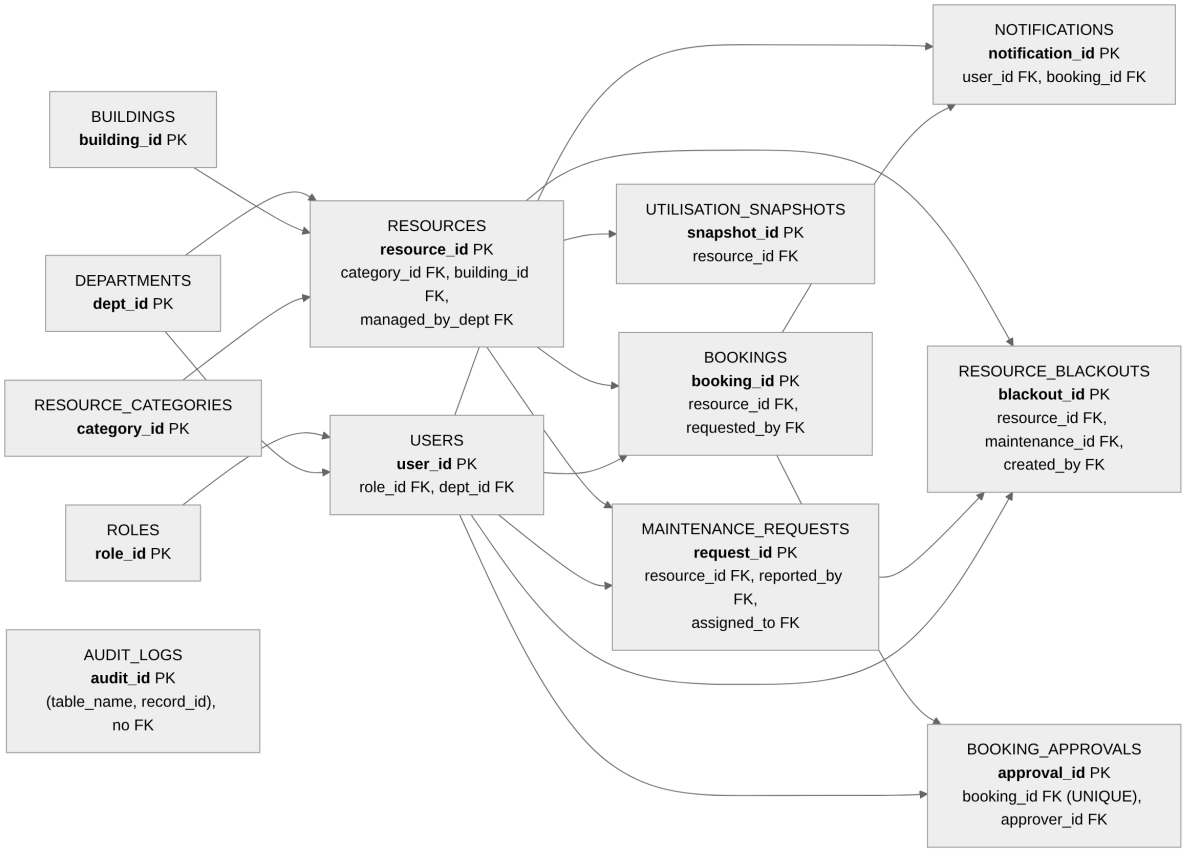


Figure 3.3: Relational schema: primary keys and foreign-key references between the thirteen tables

3.4 Schema Definitions with Constraints

Tables are created in `01_create_schema.sql` with primary keys, `NOT NULL` and `DEFAULT`; all other constraints are added with `ALTER TABLE` in `02_constraints_indexes.sql`, which exercises the full DDL syllabus (`CREATE`, `ALTER`, `DROP`). Listing 3.1 shows the central BOOKINGS table.

```
CREATE TABLE bookings (
    booking_id     NUMBER(12)     GENERATED BY DEFAULT AS IDENTITY,
    resource_id    NUMBER(10)     NOT NULL,
    requested_by   NUMBER(10)     NOT NULL,
    title          VARCHAR2(150)  NOT NULL,
```

```
    attendees      NUMBER(5)      DEFAULT 1 NOT NULL,
    start_time     TIMESTAMP      NOT NULL,
    end_time       TIMESTAMP      NOT NULL,
    status         VARCHAR2(12)   DEFAULT 'PENDING' NOT NULL,
    request_key    VARCHAR2(64),              -- idempotency key
    cancelled_at   TIMESTAMP,
    ...
    CONSTRAINT pk_bookings PRIMARY KEY (booking_id));

ALTER TABLE bookings ADD CONSTRAINT ck_bookings_interval CHECK (end_time > start_time);
ALTER TABLE bookings ADD CONSTRAINT ck_bookings_status
    CHECK (status IN ('PENDING','APPROVED','REJECTED','CANCELLED'));
ALTER TABLE bookings ADD CONSTRAINT ck_bookings_cancel
    CHECK ((status = 'CANCELLED' AND cancelled_at IS NOT NULL)
        OR (status <> 'CANCELLED' AND cancelled_at IS NULL));
ALTER TABLE bookings ADD CONSTRAINT uq_bookings_request_key UNIQUE (request_key);
ALTER TABLE bookings ADD CONSTRAINT fk_bookings_resource
    FOREIGN KEY (resource_id) REFERENCES resources (resource_id);
```

Listing 3.1: BOOKINGS table and its constraints (excerpt)

Table 3.2: Constraint inventory

Type	Count	Examples and purpose
PRIMARY KEY	13	one per table; identity columns except the fixed role IDs
FOREIGN KEY	18	named `fk_*`; `ON DELETE CASCADE` only for dependent data (approvals, notifications, snapshots), `ON DELETE SET NULL` for optional links
UNIQUE	11	e-mail, registration number, codes, request key, one decision per booking, one snapshot per resource and period
CHECK	29	interval order, status domains, cancelled $\Leftrightarrow$ `cancelled_at`, resolved $\Leftrightarrow$ `resolved_at`, operating hours, capacity ≥ 0, e-mail pattern (`REGEXP_LIKE`), `features_json IS JSON`
NOT NULL / DEFAULT	–	every mandatory column; defaults for statuses, flags and timestamps

Rules that depend on *other rows* — overlap, approval rights, maintenance state — cannot be expressed as `CHECK` constraints in Oracle. They are enforced by stored procedures and triggers (Chapter 4).

3.5 Normalisation

Figure 3.4 summarises the decomposition from an unnormalised booking register to the final schema.

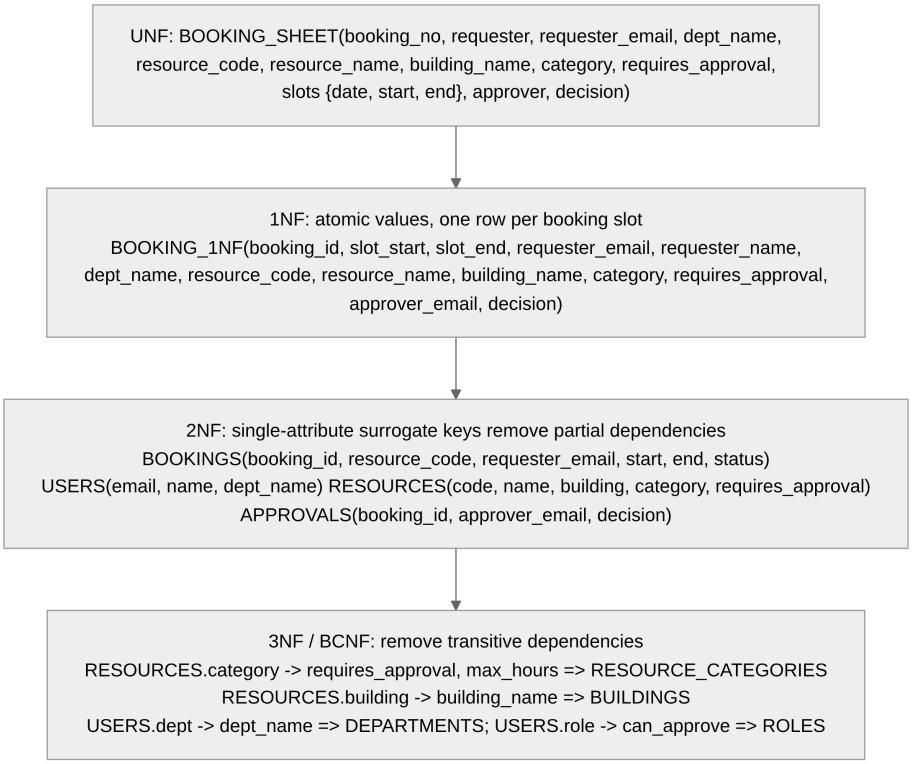


Figure 3.4: Normalisation steps from the unnormalised booking register to 3NF/BCNF

3.5.1 Unnormalised form

A typical paper or spreadsheet register records, per request, the requester's name, e-mail and department, the resource code, name, building and category, the category's approval rule, a *list* of requested dates and times, and the approver's details and decision:

`BOOKING_SHEET(booking_no, requester_name, requester_email, requester_dept, resource_code, resource_name, building_name, category, requires_approval, max_hours, {date, start, end}, approver_email, approver_name, decision)`

The braces mark a repeating group.

3.5.2 First normal form

The repeating group is removed so that every attribute is atomic and each row describes exactly one interval; each interval becomes its own booking with key `booking_id`:

`BOOKING_1NF(booking_id, start_time, end_time, requester_email, requester_name, requester_dept, dept_name, resource_code, resource_name, building_code, building_name, category, requires_approval, max_hours, approver_email, approver_name, decision)`

3.5.3 Functional dependencies

Table 3.3: Functional dependencies identified in BOOKING_1NF

Determinant	Dependent attributes
booking_id	start_time, end_time, requester_email, resource_code, status, approver_email, decision
requester_email	requester_name, requester_dept, role
requester_dept	dept_name
role	can_approve, can_administer
resource_code	resource_name, building_code, category, capacity, operating hours, managing department
building_code	building_name, campus_zone
category	requires_approval, max_hours, min_notice_hours
approver_email	approver_name

3.5.4 Second normal form

With the single-attribute key `booking_id` there is no partial dependency, so BOOKING_1NF is formally in 2NF. Had the natural composite key `(resource_code, start_time)` been used, `resource_name`, `building_code` and `category` would depend on only part of it — a partial dependency. In both cases the entity data is separated into `BOOKINGS`, `USERS`, `RESOURCES` and `APPROVALS`, each with facts about one thing only.

3.5.5 Third normal form

Transitive dependencies (non-key determining non-key) remain in the 2NF relations and are removed by decomposition:

- resource_code → category → requires_approval, max_hours gives RESOURCE_CATEGORIES;
- resource_code → building_code → building_name gives BUILDINGS;
- email → dept → dept_name gives DEPARTMENTS;
- email → role → can_approve gives ROLES.

Without this step, changing a category's approval rule would require updating every resource row (update anomaly), and a category with no resources could not be recorded (insertion anomaly).

3.5.6 Boyce–Codd normal form

A relation is in BCNF when every non-trivial determinant is a candidate key. In the final schema the only determinants are the surrogate keys and the declared unique keys (`email`, `reg_no`, `resource_code`, the reference-table codes, `request_key`, `booking_id` in BOOKING_APPROVALS and `(resource_id, period_start, period_end)` in UTILISATION_SNAPSHOTS), all of which are candidate keys. The pair `(resource_id, start_time)` is deliberately *not* unique: a cancelled or rejected booking may share a slot with a later live booking, so uniqueness of live slots is an overlap rule rather than a functional dependency. Every relation is therefore in BCNF. Derived values — duration and utilisation — are not stored; they are computed by functions and views to avoid update anomalies.

3.6 Indexes

Oracle does not create indexes for foreign keys automatically. An unindexed foreign key causes full scans on joins and can lead to table-level locks on the child table when the parent key is modified. Twenty-two explicit indexes cover every foreign key (checked by query V5 in `13_verification.sql`) and the three most frequent access paths:

Table 3.4: Access-path indexes

Index	Query it serves
ix_bookings_res_time (resource_id, start_time, end_time)	overlap check inside the booking procedure; calendar; utilisation
ix_bookings_user_start (requested_by, start_time)	"My bookings"
ix_bookings_status_start (status, start_time)	approval queue; dashboard counts
ix_maint_status_priority, ix_maint_resource	maintenance board
ix_audit_record (table_name, record_id)	change history of one booking
ix_notif_user_read (user_id, is_read, created_at)	notification bell

4. Implementation

Table 4.1 lists every database object. The scripts are numbered in execution order and `run_all.sql` installs them and stops if any object is invalid.

Table 4.1: Database objects implemented

Type	Objects
Tables (13)	see Table 3.1
Views (9)	v_resource_catalog, v_booking_details, v_pending_approvals, v_active_bookings, v_maintenance_dashboard, v_resource_utilisation_30d, v_department_usage, v_booking_trend_daily, v_top_resources
Functions (5)	fn_booking_duration_hours, fn_resource_utilisation, fn_resource_availability; helpers fn_hours_between, fn_current_actor
Procedures (5)	sp_create_booking, sp_decide_booking, sp_cancel_booking, sp_submit_maintenance, sp_update_maintenance_status
Packages (2)	pkg_campus_reservations, pkg_resource_analytics (specification and body)
Triggers (9)	trg_users_biu, trg_resources_bu, trg_bookings_biu, trg_bookings_no_overlap (compound), trg_bookings_audit, trg_maintenance_biu, trg_maintenance_audit, trg_resources_status_audit, trg_audit_logs_immutable
Cursors	explicit (send_pending_digest), explicit `FOR UPDATE SKIP LOCKED` (expire_stale_pending), cursor `FOR` loops (escalate_stale_maintenance, sp_submit_maintenance, print_utilisation_report), parameterised cursor (build_utilisation_snapshot), REF cursors (seven package functions)

4.1 SQL Coverage of the Syllabus

The indicative experiments of the BCSE302P syllabus map to the implementation as shown in Table 4.2.

Table 4.2: Syllabus experiments and where they are demonstrated

Syllabus experiment	Demonstration in CampusReserve
1. DDL and DML	`CREATE/ALTER/DROP` (01, 02, 00); `INSERT`, `UPDATE`, `DELETE` (seed, procedures, tests); `MERGE` (snapshot procedure)
2. Constraints	Table 3.2; violation tests TC-17 and TC-18
3. Single-row functions	`UPPER`, `LOWER`, `TRIM`, `NVL`, `NVL2`, `ROUND`, `TRUNC`, `LPAD`, `RPAD`, `SUBSTR`, `TO_CHAR`, `EXTRACT`, `CAST`, `GREATEST`, `LEAST`, `NUMTODSINTERVAL`, `REGEXP_LIKE`, `CASE`
4. Operators and group functions	`IN`, `BETWEEN`, `LIKE`, comparison and interval operators; `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`, `COUNT(DISTINCT)`; `GROUP BY/HAVING` in v_department_usage; `DENSE_RANK` and `COUNT(*) OVER()` analytic functions
5. Subqueries, views and joins	scalar and correlated subqueries (v_resource_catalog, v_pending_approvals), `NOT EXISTS` (verification V5), inner, left outer, cross and self joins (overlap self-join V7), `UNION ALL` (resource_schedule), nine views
6. Procedures, functions, cursors, triggers	Sections 4.2–4.6

Listing 4.1 shows a view that combines a left outer join, aggregates and `HAVING`.

```
CREATE OR REPLACE VIEW v_department_usage AS
SELECT d.dept_code, d.dept_name,
       COUNT(bk.booking_id)                                        AS total_requests,
       SUM(CASE WHEN bk.status = 'APPROVED' THEN 1 ELSE 0 END)  AS approved,
       ROUND(SUM(CASE WHEN bk.status = 'APPROVED'
                      THEN (CAST(bk.end_time AS DATE) - CAST(bk.start_time AS DATE)) * 24
                      ELSE 0 END), 1)                              AS approved_hours,
       COUNT(DISTINCT bk.resource_id)                              AS distinct_resources
FROM departments d
JOIN users u     ON u.dept_id = d.dept_id
JOIN bookings bk ON bk.requested_by = u.user_id
GROUP BY d.dept_id, d.dept_code, d.dept_name
HAVING COUNT(bk.booking_id) > 0;
```

Listing 4.1: Department usage view with GROUP BY and HAVING

4.2 Functions

Functions have no side effects so they can be called from SQL as well as PL/SQL.

Table 4.3: Functions: parameters and results

Function	Input	Output
fn_booking_duration_hours	p_booking_id	hours (2 dp); ORA-20010 if missing
fn_resource_utilisation	p_resource_id, p_from (incl.), p_to (excl.)	percentage of operating hours booked
fn_resource_availability	p_resource_id, p_start, p_end, p_ignore_booking	AVAILABLE, INVALID_INTERVAL, NOT_FOUND, MAINTENANCE, RETIRED, OUTSIDE_HOURS, BLACKOUT or CONFLICT

Utilisation logic. Approved hours are summed only for bookings that intersect the window, and each booking is clipped to the window with `GREATEST/LEAST` so that a booking straddling the boundary is counted only for the part inside. The denominator is the number of days in the window multiplied by the resource's daily operating hours.

```
SELECT NVL(SUM(fn_hours_between(GREATEST(start_time, v_from_ts),
                                LEAST(end_time, v_to_ts))), 0)
  INTO v_booked
  FROM bookings
 WHERE resource_id = p_resource_id AND status = 'APPROVED'
   AND start_time < v_to_ts AND end_time > v_from_ts;
v_available := (TRUNC(p_to) - TRUNC(p_from)) * (v_close - v_open);
RETURN LEAST(100, ROUND(100 * v_booked / v_available, 2));
```

Listing 4.2: Utilisation function (core)

Availability logic. Two half-open intervals $[s_1, e_1)$ and $[s_2, e_2)$ overlap if and only if $s_1 < e_2 \wedge s_2 < e_1$ [7]. Using half-open intervals means a booking ending at 12:00 and another starting at 12:00 do not conflict, which test TC-06b checks. The function is *advisory*: it powers the live "This time is free" indicator in the booking form, but the final decision is always made by `sp_create_booking` under a lock.

Example. `SELECT fn_resource_availability(1, TIMESTAMP '2026-11-20 10:00:00', TIMESTAMP '2026-11-20 12:00:00') FROM dual;` is expected to return `AVAILABLE` on an empty slot and `CONFLICT` after a booking is placed there. Actual output: Figure 4.1.

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig08_functions.png` and rebuild.

Figure 4.1: Function outputs from verification script section V8 (own execution)

4.3 Stored Procedures and Transaction-safe Booking

Table 4.4: Stored procedures

Procedure	IN parameters	OUT / effect
sp_create_booking	resource, user, title, purpose, attendees, start, end, request key	o_booking_id, o_status (PENDING/APPROVED), o_duplicate
sp_decide_booking	booking, approver, decision, remarks	status change, approval row, notification
sp_cancel_booking	booking, user, reason	CANCELLED, slot freed, notification
sp_submit_maintenance	resource, user, title, description, priority, blocks_booking	o_request_id; resource set to MAINTENANCE if blocking; holders notified
sp_update_maintenance_status	request, actor, new status, assignee, notes	ticket updated; resource status recomputed

4.3.1 Why a simple overlap check is not enough

A naive implementation runs `SELECT COUNT(*)` for overlapping bookings and inserts when the count is zero. Under Oracle's default `READ COMMITTED` isolation each statement sees only data committed before it began. If sessions A and B both run the check before either commits, both see zero and both insert; the overlap is committed. This is the classic phantom problem described by Eswaran et al. [5] and analysed for SQL isolation levels by Berenson et al. [6]: the conflicting row does not yet exist, so there is nothing for an ordinary row lock to protect.

4.3.2 Concurrency-safe design

`sp_create_booking` serialises all writers *per resource* by locking the parent `RESOURCES` row before checking (Listing 4.3). The resource row acts as a lock object for the set of its bookings, standing in for the predicate lock that Oracle does not provide. The second session blocks at the `SELECT ... FOR UPDATE`; when the first commits, the second resumes, and its overlap

query — a *new* statement — sees the committed booking and raises ORA-20005 (Figure 4.2). Requests for *different* resources never wait for each other. `WAIT 5` turns an unusually long wait into ORA-20018 ("resource busy, retry") instead of hanging the API.

```
-- (d) SERIALISATION POINT: lock the resource row.
SELECT r.status, r.capacity, r.opens_at_hour, r.closes_at_hour, ...
  INTO v_res_status, v_capacity, v_open, v_close, ...
  FROM resources r
  JOIN resource_categories c ON c.category_id = r.category_id
 WHERE r.resource_id = p_resource_id
   FOR UPDATE OF r.status WAIT 5;
-- (e) resource rules: maintenance, capacity, hours, max duration, notice, blackout
IF v_res_status = 'MAINTENANCE' THEN
    RAISE_APPLICATION_ERROR(-20003, v_res_name || ' is under maintenance ...');
END IF;
...
-- (f) overlap check, made while holding the lock
SELECT MIN(booking_id) INTO v_conflict_id
  FROM bookings
 WHERE resource_id = p_resource_id
   AND status IN ('PENDING', 'APPROVED')
   AND start_time < p_end AND end_time > p_start;
IF v_conflict_id IS NOT NULL THEN
    RAISE_APPLICATION_ERROR(-20005, 'Time slot conflicts with booking #' || v_conflict_id
     || '.');
END IF;
-- (g) accept: no COMMIT here - the caller owns the transaction
INSERT INTO bookings (...) VALUES (...) RETURNING booking_id INTO o_booking_id;
```

Listing 4.3: sp_create_booking: serialisation point and overlap check (excerpt)

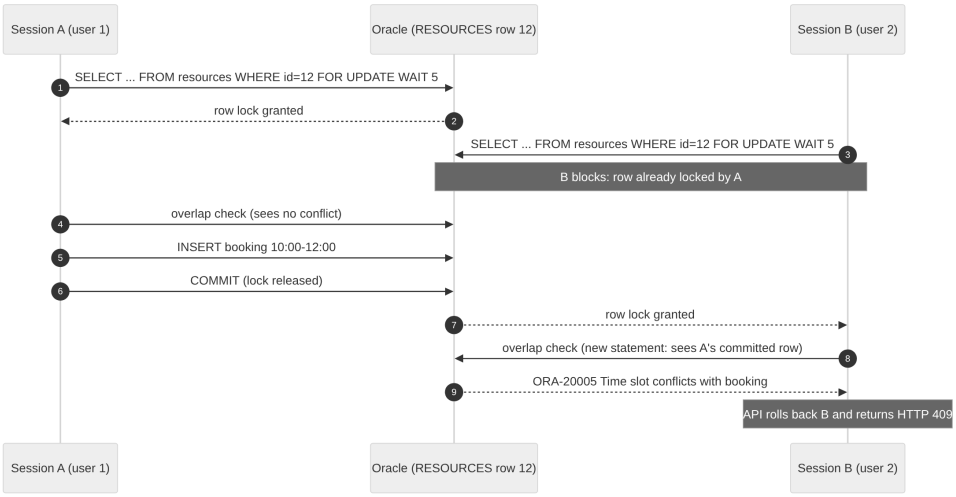


Figure 4.2: Two sessions competing for the same slot: the row lock serialises them

Further safeguards:

- **Lock ordering.** Every procedure locks RESOURCES before BOOKINGS or MAINTENANCE_REQUESTS, which removes the circular wait needed for a deadlock.
- **No commits in procedures.** The procedures never `COMMIT` or `ROLLBACK`. The API wraps

each request in one transaction (Listing 4.4), so a multi-step operation is atomic and the SQL test scripts can roll back everything they do.

- **Idempotency.** The client sends a random `Idempotency-Key`; a retried request returns the original booking instead of creating a duplicate (TC-18g). A race between two retries is caught by the unique constraint (`DUP_VAL_ON_INDEX`).
- **Authorisation in the database.** `sp_decide_booking` checks that the approver's role can approve and, for coordinators, that the resource belongs to their department.

```
export async function withTransaction(actorId, fn) {
  const conn = await pool.getConnection();
  conn.clientId = actorId ? String(actorId) : '';   // read by audit triggers
  try {
    const result = await fn(conn);                  // e.g. BEGIN sp_create_booking(...);
     END;
    await conn.commit();                            // one commit per API request
    return result;
  } catch (err) {
    await conn.rollback();                          // any failure undoes the whole unit
    throw err;
  } finally {
    await conn.close();
  }
}
```

Listing 4.4: API transaction wrapper (backend/src/config/db.js, excerpt)

Execution example (SQL*Plus).

```
VARIABLE id NUMBER
VARIABLE st VARCHAR2(12)
VARIABLE dup NUMBER
EXEC sp_create_booking(1, 7, 'DBMS project review', NULL, 12, -
     TIMESTAMP '2026-11-20 10:00:00', TIMESTAMP '2026-11-20 12:00:00', NULL, :id, :st, :
    dup);
PRINT id st
```

Listing 4.5: Calling sp_create_booking

Expected output: a new booking ID and status `PENDING` (computer laboratories require approval). Repeating the call returns ORA-20005. Actual output is part of the test-suite evidence in Chapter 6.

4.4 Packages

Each package has a separate specification (`07_package_specifications.sql`) and body (`08_package_bodies.sql`).

PKG_CAMPUS_RESERVATIONS holds the reservation workflow helpers: status constants; named exceptions bound to the application error codes with `PRAGMA EXCEPTION_INIT` (so PL/SQL callers can write `WHEN pkg_campus_reservations.e_booking_conflict`);

the pipelined function `day_slots` that generates the hourly availability strip; the REF-cursor functions `resource_schedule` (calendar, using `UNION ALL` of bookings and blackouts) and `pending_for_approver`; and the cursor procedures `send_pending_digest` and `expire_stale_pending`.

PKG_RESOURCE_ANALYTICS computes everything shown on the analytics page: `dashboard_summary`, `booking_trend` (with a `CONNECT BY` row generator so days with no bookings still appear), `top_resources`, `department_usage`, `resource_utilisation`, `peak_hours`, plus `build_utilisation_snapshot`, `escalate_stale_maintenance` and `print_utilisation_report`.

```
CREATE OR REPLACE PACKAGE pkg_campus_reservations AS
    c_pending  CONSTANT VARCHAR2(12) := 'PENDING';
    e_booking_conflict EXCEPTION; PRAGMA EXCEPTION_INIT(e_booking_conflict, -20005);
    TYPE t_slot IS RECORD (slot_start TIMESTAMP, slot_end TIMESTAMP,
                           slot_state VARCHAR2(12), booking_id NUMBER);
    TYPE t_slot_tab IS TABLE OF t_slot;
    FUNCTION day_slots (p_resource_id IN NUMBER, p_day IN DATE,
                        p_slot_minutes IN NUMBER DEFAULT 60) RETURN t_slot_tab PIPELINED;
    FUNCTION resource_schedule (p_resource_id IN NUMBER, p_from IN TIMESTAMP,
                                p_to IN TIMESTAMP) RETURN SYS_REFCURSOR;
    FUNCTION pending_for_approver (p_approver_id IN NUMBER) RETURN SYS_REFCURSOR;
    PROCEDURE send_pending_digest  (p_approver_id IN NUMBER, o_pending_count OUT NUMBER);
    PROCEDURE expire_stale_pending (p_actor_id IN NUMBER, o_expired OUT NUMBER);
END pkg_campus_reservations;
```

Listing 4.6: Package specification (excerpt)

Execution example. `SELECT * FROM TABLE(pkg_campus_reservations.day_slots(12, TRUNC(SYSDATE)+1, 60));` returns one row per hour with state FREE, BOOKED, PENDING, BLACKOUT, PAST or CLOSED; for a resource open 08:00–20:00 it returns 12 rows (TC-22).

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig08_refcursor.png` and rebuild.

Figure 4.3: REF cursor output of pkg_resource_analytics.dashboard_summary and pipelined day_slots (own execution)

4.5 Triggers

Table 4.5: Triggers

Trigger	Timing	Purpose
trg_bookings_biu	before insert/update, row	new rows must be PENDING/APPROVED on an AVAILABLE resource within capacity; status state machine; sets `cancelled_at`; blocks rescheduling of final bookings
trg_bookings_no_overlap	compound	defence-in-depth overlap detection (below)
trg_bookings_audit	after insert/update/delete, row	audit row with old/new status and details
trg_maintenance_biu	before insert/update, row	ticket state machine; maintains `resolved_at`
trg_maintenance_audit	after insert/update, row	audit of status, priority and assignment
trg_resources_status_audit	after update of status, row, `WHEN`	audit of availability changes
trg_resources_bu, trg_users_biu	before, row	`updated_at`; lower-case e-mail
trg_audit_logs_immutable	before update/delete, statement	makes AUDIT_LOGS append-only

Avoiding mutating-table errors. A row-level trigger on BOOKINGS may not query BOOKINGS (ORA-04091). The overlap trigger is therefore a *compound* trigger: the row section only remembers the affected IDs, and the check runs in the `AFTER STATEMENT` section, when the table is no longer mutating (Listing 4.7). Row-level triggers only read *other* tables, which is permitted.

```
CREATE OR REPLACE TRIGGER trg_bookings_no_overlap
FOR INSERT OR UPDATE OF start_time, end_time, status, resource_id ON bookings
COMPOUND TRIGGER
    TYPE t_ids IS TABLE OF bookings.booking_id%TYPE;
    g_ids t_ids := t_ids();
    AFTER EACH ROW IS
    BEGIN
        IF :NEW.status IN ('PENDING', 'APPROVED') THEN
            g_ids.EXTEND; g_ids(g_ids.LAST) := :NEW.booking_id;
        END IF;
    END AFTER EACH ROW;
    AFTER STATEMENT IS
        v_conflict bookings.booking_id%TYPE;
    BEGIN
        FOR i IN 1 .. g_ids.COUNT LOOP
            SELECT MIN(o.booking_id) INTO v_conflict
              FROM bookings b JOIN bookings o
                ON o.resource_id = b.resource_id AND o.booking_id <> b.booking_id
```

```
              AND o.status IN ('PENDING','APPROVED')
              AND o.start_time < b.end_time AND o.end_time > b.start_time
            WHERE b.booking_id = g_ids(i);
           IF v_conflict IS NOT NULL THEN
               RAISE_APPLICATION_ERROR(-20017, 'Booking #' || g_ids(i) || ' overlaps #'
    || v_conflict);
           END IF;
       END LOOP;
    END AFTER STATEMENT;
END trg_bookings_no_overlap;
```

Listing 4.7: Compound overlap trigger

This trigger catches overlaps written by *any* client, for example an ad-hoc `INSERT` (TC-18h). It cannot by itself stop two concurrent transactions, because each session's check cannot see the other's uncommitted row; that limitation is exactly why the procedure holds the resource lock, and it is demonstrated experimentally in Section 6.4.

Audit attribution. The API sets `connection.clientId` to the signed-in user's ID; triggers read it through `fn_current_actor` (`SYS_CONTEXT('USERENV','CLIENT_IDENTIFIER')`) and fall back to the row's own user column in SQL*Plus sessions.

4.6 Cursors

- **Explicit cursor — pending approvals.** `send_pending_digest` declares a parameterised cursor over the approver's queue and uses `OPEN`, `FETCH`, `%NOTFOUND`, `%ROWCOUNT` and `CLOSE` to build one digest notification listing the first five requests (Listing 4.8). The exception handler closes the cursor if it is still open (`%ISOPEN`).
- **Explicit cursor with `FOR UPDATE SKIP LOCKED`.** `expire_stale_pending` rejects pending requests whose start time has passed, updating each row with `WHERE CURRENT OF`; rows another session is deciding at that moment are skipped rather than waited for.
- **Cursor `FOR` loop — maintenance requests.** `escalate_stale_maintenance` iterates over tickets left OPEN longer than n days, raises their priority with `WHERE CURRENT OF` and alerts administrators. `sp_submit_maintenance` uses an implicit cursor `FOR` loop to notify holders of upcoming bookings.
- **Cursor-based reporting.** `build_utilisation_snapshot` loops over a parameterised cursor that aggregates approved hours per resource and `MERGE`s one row per resource into UTILISATION_SNAPSHOTS; `print_utilisation_report` prints a formatted text report through `DBMS_OUTPUT`.
- **REF cursors.** Seven package functions return `SYS_REFCURSOR`; node-oracledb reads them as result sets.

```
CURSOR c_queue (cp_dept NUMBER, cp_is_admin CHAR) IS
```

```
    SELECT p.booking_id, p.resource_code, p.start_time, p.requester_name
      FROM v_pending_approvals p JOIN resources r ON r.resource_id = p.resource_id
     WHERE cp_is_admin = 'Y' OR r.managed_by_dept = cp_dept
     ORDER BY p.start_time;
v_rec c_queue%ROWTYPE;
BEGIN
    OPEN c_queue(v_dept, v_is_admin);
    LOOP
        FETCH c_queue INTO v_rec;
        EXIT WHEN c_queue%NOTFOUND;
        IF c_queue%ROWCOUNT <= 5 THEN
            v_message := v_message || CASE WHEN v_message IS NOT NULL THEN '; ' END
                      || '#' || v_rec.booking_id || ' ' || v_rec.resource_code;
        END IF;
    END LOOP;
    o_pending_count := c_queue%ROWCOUNT;
    CLOSE c_queue;
```

Listing 4.8: Explicit cursor in send_pending_digest (excerpt)

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig08_cursor_report.png` and rebuild.

Figure 4.4: Cursor FOR loop utilisation report printed with DBMS_OUTPUT (own execution)

5. Front-End and Database Integration

5.1 Integration Architecture

The front end is a React and TypeScript single-page application built with Vite. It never contains database credentials and never builds SQL. It calls the REST API with a bearer token; the API maps each call onto a database object (Table 5.1), always with bind variables. Oracle application errors are translated into HTTP status codes and readable messages, so a conflict detected inside `sp_create_booking` reaches the user as "Time slot conflicts with booking #412".

Table 5.1: Screens and the database objects behind them

Screen	Database object used
Sign-in / registration	USERS, ROLES, DEPARTMENTS (bcrypt check in API)
Overview dashboard	pkg_resource_analytics.dashboard_summary, booking_trend; v_booking_details
Find a space, resource detail	v_resource_catalog; fn_resource_utilisation
Availability calendar	pkg_campus_reservations.resource_schedule (REF cursor)
New booking	fn_resource_availability (live check), day_slots (pipelined), sp_create_booking
My bookings, booking detail	v_booking_details; AUDIT_LOGS history; sp_cancel_booking
Approvals	pending_for_approver; sp_decide_booking; send_pending_digest; expire_stale_pending
Maintenance board	v_maintenance_dashboard; sp_submit_maintenance; sp_update_maintenance_status
Analytics	resource_utilisation, top_resources, department_usage, peak_hours; build_utilisation_snapshot
Resource administration	RESOURCES, RESOURCE_BLACKOUTS
Users, audit history, profile	USERS; AUDIT_LOGS

5.2 Interface Design

The interface uses a restrained glass style: translucent panels over a soft mist background, Manrope headings and Inter body text with system-font fallbacks, a single teal action colour and consistent status colours (green approved, amber pending, red rejected, violet maintenance). The availability calendar draws bookings as frosted time blocks and pending requests with dashed borders, and the booking form highlights the requested interval. Every data view has loading, empty and error states; all controls are keyboard-accessible with visible focus rings; motion is disabled when the user prefers reduced motion. Layouts were checked at 390, 820 and 1440 pixel widths.

5.3 Screenshots

The following figures must be captured from your own running system (Evidence Capture Guide, §3).

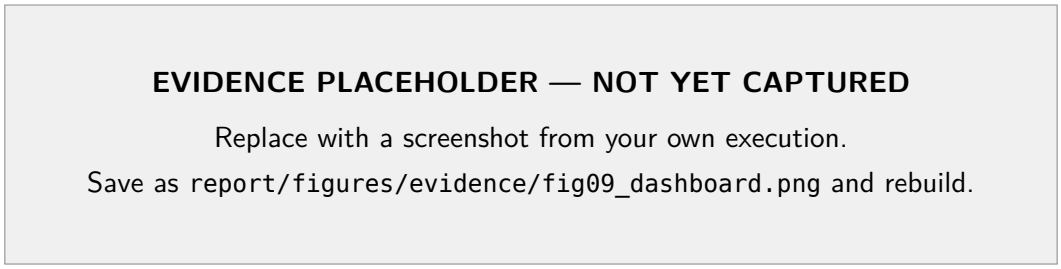


Figure 5.1: Overview dashboard with database-backed KPI tiles and booking trend

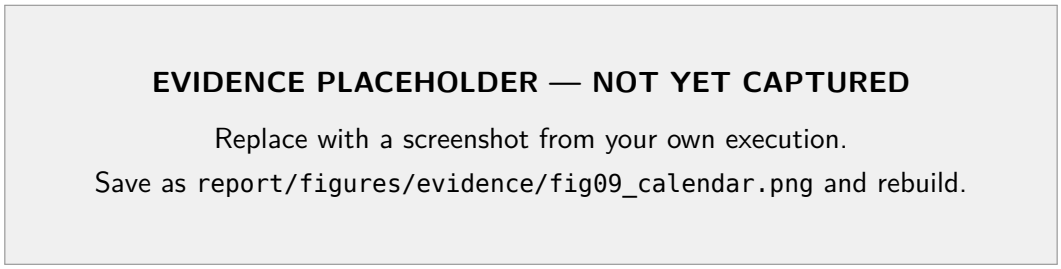


Figure 5.2: Availability calendar fed by the resource_schedule REF cursor

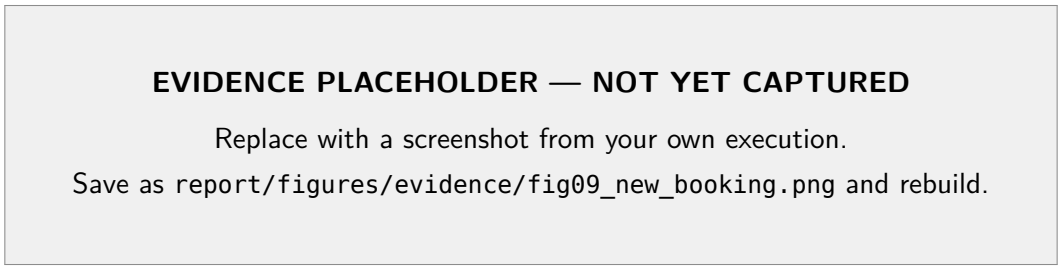


Figure 5.3: New booking form: live availability check and pipelined hourly slots



Figure 5.4: Overlap rejected by sp_create_booking and shown to the user (HTTP 409)

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig09_approvals.png` and rebuild.

Figure 5.5: Approval queue for a coordinator

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig09_maintenance.png` and rebuild.

Figure 5.6: Maintenance board

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig09_analytics.png` and rebuild.

Figure 5.7: Analytics computed by PKG_RESOURCE_ANALYTICS

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig09_booking_detail.png` and rebuild.

Figure 5.8: Booking detail with trigger-written change history

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig09_mobile.png` and rebuild.

Figure 5.9: Responsive layout at 390 px width

6. Verification and Testing

6.1 Test Strategy

Testing is layered. (1) `10_test_cases.sql` is a self-checking PL/SQL suite: each case prints PASS or FAIL with the observed result, and the whole run is rolled back so it can be repeated. (2) `12_transaction_tests.sql` demonstrates statement atomicity, savepoints and multi-step rollback, and gives a two-session protocol for concurrency. (3) `11_query_optimization.sql` records execution plans and timings before and after indexing. (4) The API has unit tests that need no database and an integration suite that runs against Oracle. (5) `concurrency-experiment.js` fires parallel sessions at one slot.

6.2 Execution Status

Table 6.1: Execution status of each test suite at the time of writing

Suite	Status
API unit and HTTP-layer tests (25)	executed: 25 passed, 0 failed
Front-end type check, lint, production build	executed: no errors
Database functional suite (TC-04 to TC-22)	*Pending: run 10_test_cases.sql*
Transaction and two-session tests	*Pending: run 12_transaction_tests.sql*
Execution-plan experiment	*Pending: run 11_query_optimization.sql*
API integration suite against Oracle	*Pending: RUN_DB_TESTS=1 npm test*
Concurrency experiment	*Pending: node scripts/concurrency-experiment.js*

6.3 Test Cases

In Table 6.2 the *Actual* column must contain only results observed in your own execution; until then it reads *Pending*.

Table 6.2: Functional test cases

ID	Objective / input	Expected result	Actual	Status
TC-01	Registration with valid data	201, role STUDENT, token	*Pending:*	–
TC-02	Login, seeded user	200, token, dashboard	*Pending:*	–
TC-03	Login, wrong password	401 INVALID_CREDENTIALS	*Pending:*	–

ID	Objective / input	Expected result	Actual	Status
TC-04	Create resource	row created, status AVAILABLE	*Pending:*	–
TC-05	Book free lab 10:00–12:00	ID returned, PENDING, 2 h	*Pending:*	–
TC-06	Book 11:00–13:00 same lab	ORA-20005 / 409; 12:00–14:00 accepted	*Pending:*	–
TC-07	10 simultaneous requests	locked path: exactly one success	*Pending:*	–
TC-08	Coordinator approves	APPROVED + approval row; re-decision ORA-20011	*Pending:*	–
TC-09	Coordinator rejects with reason	REJECTED; slot available	*Pending:*	–
TC-10	Owner cancels	CANCELLED, slot free; non-owner ORA-20012	*Pending:*	–
TC-11	Book resource under maintenance	ORA-20003 / 409	*Pending:*	–
TC-12	Student approves / calls admin API	ORA-20012; HTTP 403	HTTP layer: 403 returned in unit tests; DB: *Pending:*	partly executed
TC-13	Blocking maintenance ticket	OPEN; resource MAINTENANCE	*Pending:*	–
TC-14	Ticket to RESOLVED	resolved_at set; resource AVAILABLE; CLOSED→OPEN ORA-20011	*Pending:*	–
TC-15	Audit generation; delete audit row	$\geq$4 ticket rows; ORA-20012 on delete	*Pending:*	–
TC-16	Utilisation function vs SQL	equal values	*Pending:*	–
TC-17	Invalid foreign key	ORA-02291	*Pending:*	–
TC-18	Constraint violations (8 sub-cases)	ORA-02290, ORA-00001, -20001, -20006, -20008, -20007, idempotent retry, -20017	*Pending:*	–
TC-19	Rollback to savepoint	row count unchanged	*Pending:*	–
TC-20	Plans before/after index	FULL scan → INDEX RANGE SCAN	*Pending:*	–
TC-21	Cursor procedures	counts printed; snapshot rows = resources	*Pending:*	–

ID	Objective / input	Expected result	Actual	Status
TC-22	Pipelined slots	12 slots for 08:00–20:00	*Pending:*	–

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig10_test_suite_1.png` and rebuild.

Figure 6.1: Output of 10_test_cases.sql: PASS/FAIL lines (part 1)

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig10_test_suite_2.png` and rebuild.

Figure 6.2: Output of 10_test_cases.sql: remaining cases and totals

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig10_unit_tests.png` and rebuild.

Figure 6.3: API unit tests (npm test) in the terminal

6.4 Experiment E2: Simultaneous Booking Attempts

Method. Ten independent database sessions attempt to book the same resource for the same interval at the same instant, first with a naive check-then-insert block (with a 0.5 s sleep between check and insert to widen the race window) and then through `sp_create_booking`. The script records how many sessions succeed and how many live rows end up in the slot. Each mode uses a separate empty slot, and all rows are cancelled afterwards.

Expected behaviour. Locked path: exactly one success, nine ORA-20005 conflicts, one live row. Naive path: more than one success is possible; some late sessions may be stopped by the compound trigger (ORA-20017) if an earlier session has already committed.

Table 6.3: Concurrency experiment results (record every run)

Mode	Run	Sessions succeeded	Live rows in slot	Elapsed (ms)
Naive	1	*Pending:*	*Pending:*	*Pending:*
Naive	2	*Pending:*	*Pending:*	*Pending:*
Naive	3	*Pending:*	*Pending:*	*Pending:*
Locked	1	*Pending:*	*Pending:*	*Pending:*
Locked	2	*Pending:*	*Pending:*	*Pending:*
Locked	3	*Pending:*	*Pending:*	*Pending:*

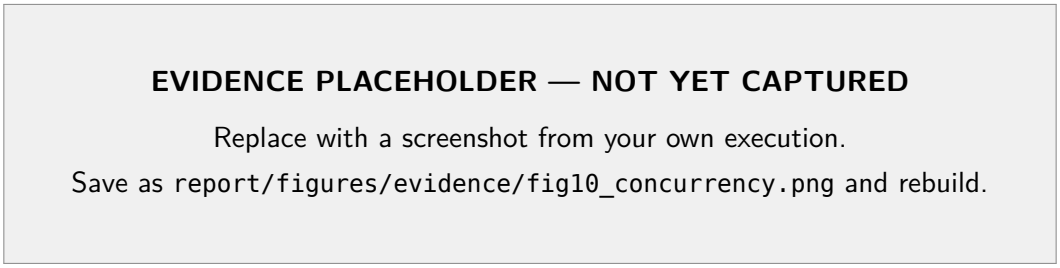


Figure 6.4: Output of concurrency-experiment.js (naive versus locked)

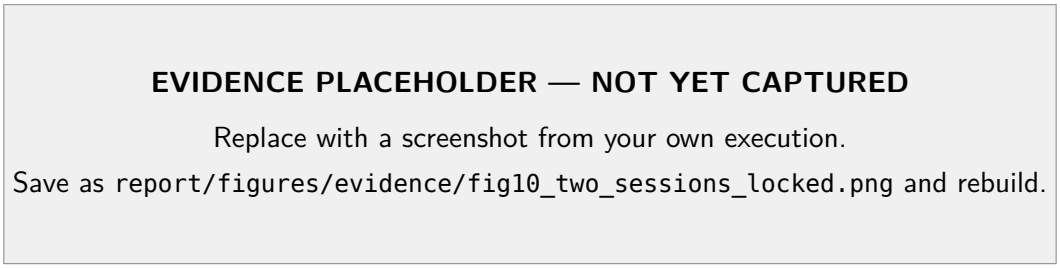


Figure 6.5: Two SQL sessions: session B blocks until A commits, then receives ORA-20005

6.5 Transaction Rollback

Part A of `12_transaction_tests.sql` shows that (A1) a multi-row `INSERT` in which one row violates a unique constraint inserts nothing; (A2) `ROLLBACK TO SAVEPOINT` undoes a cancellation while keeping the earlier booking; and (A3) a two-step unit — submit a blocking maintenance ticket, then book the same hall — fails in step 2 and is rolled back completely, restoring the hall's status.

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig10_transactions.png` and rebuild.

Figure 6.6: Transaction tests A1–A3

6.6 Experiment E3: Execution Plans Before and After Indexing

Method. A synthetic table BENCH_BOOKINGS of 300 000 rows (200 resources over two years) is created without indexes and analysed with `DBMS_STATS`. For three application queries — the overlap check, a user's history and the approval queue — the script captures `EXPLAIN PLAN` output and the mean elapsed time of 200 executions with varying bind values. It then creates the same composite indexes the application uses, re-analyses, and repeats. The real BOOKINGS table is not touched.

Expected plan change. Without indexes, each query must read the whole table (`TABLE ACCESS FULL`). With `(resource_id, start_time, end_time)` the optimiser can use `INDEX RANGE SCAN` on the leading equality column and apply the time predicates within the index; similarly for `(requested_by, start_time)` and `(status, start_time)`.

Table 6.4: Execution-plan experiment (fill in from your own output)

Query	Plan before	Cost before	Plan after	Cost after
Q1 overlap check	*Pending:*	*Pending:*	*Pending:*	*Pending:*
Q2 user history	*Pending:*	*Pending:*	*Pending:*	*Pending:*
Q3 approval queue	*Pending:*	*Pending:*	*Pending:*	*Pending:*
	Mean time before (ms)		Mean time after (ms)	
Q1	*Pending:*		*Pending:*	
Q2	*Pending:*		*Pending:*	
Q3	*Pending:*		*Pending:*	

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig10_plan_before.png` and rebuild.

Figure 6.7: DBMS_XPLAN output for Q1 before indexing

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig10_plan_after.png` and rebuild.

Figure 6.8: DBMS_XPLAN output for Q1 after indexing

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig10_timings.png` and rebuild.

Figure 6.9: Timing harness output and plan cost summary

7. Results and Discussion

7.1 Achievements

- A thirteen-table schema in BCNF with 18 foreign keys, 11 unique and 29 named check constraints, and an index on every foreign key.
- A single write path for bookings and maintenance: five stored procedures that never commit, with all business rules and authorisation checks inside the database.
- Concurrency control by locking the parent resource row, with consistent lock ordering and a bounded wait.
- Defence in depth: a compound trigger that catches overlaps written outside the procedures without causing mutating-table errors.
- Two packages, five functions, nine triggers and six cursor techniques, each used by a screen or job rather than only demonstrated.
- An append-only audit trail with user attribution through the session client identifier.
- A responsive front end in which every number is computed by the database.
- Reproducible experiments for conflict prevention, concurrency, rollback and indexing.

7.2 Discussion of the Research Question

Integrity constraints make invalid states unrepresentable: an interval that ends before it starts, a cancelled booking without a cancellation time, a second decision for the same booking or a dangling reference is rejected by the DBMS for every client, including ad-hoc SQL (H2).

Row-level locking addresses what constraints cannot: the absence of a conflicting row is not a row that can be locked, so a check-then-insert sequence is vulnerable to the phantom problem [5, 6]. Locking the parent resource row turns the per-resource booking set into a serialised critical section while leaving other resources fully concurrent. Moving the logic into a stored procedure also shortens the critical section to a few statements inside the database, rather than several network round trips. Whether the naive path actually produced overlaps in your runs is reported in Table 6.3 *Pending: insert observed result*.

Indexing changes the access path from full scans to range scans for the queries the application issues most. The magnitude of the improvement must be read from Table 6.4 *Pending: insert measured ratios*; it depends on table size, selectivity and buffer-cache state.

Views and aggregate queries let analytics share the same source of truth as operations. Utilisation is computed on demand with interval clipping, and the snapshot procedure stores dated results only when a historical record is wanted.

7.3 Challenges Faced

- **Mutating-table errors.** A row trigger cannot read its own table; the overlap check was moved to the `AFTER STATEMENT` section of a compound trigger.
- **Transaction ownership.** Commits inside procedures would have made multi-step API operations non-atomic and the tests impossible to roll back; all commits were moved to the API.
- **Time zones.** Bookings are campus-local times stored in `TIMESTAMP`; the API requires ISO timestamps with an offset and runs in the Asia/Kolkata zone so that values are unambiguous.
- **Version compatibility.** Features new in 23ai (SQL `BOOLEAN`, `IF EXISTS`) were avoided, and `INSERT ... SELECT` was not combined with `RETURNING`, which earlier releases do not allow.

7.4 Limitations and Possible Improvements

- The overlap rule relies on every writer using the procedures (or accepting the trigger's best-effort check). Oracle has no declarative exclusion constraint for intervals; a materialised-view or slot-table design could make it declarative at a storage cost.
- Bookings must fit within one day; recurring bookings are not supported.
- The JWT is kept in session storage; an HTTP-only cookie with CSRF protection would be stronger when front end and API share a domain.
- Notifications are in-app only; e-mail delivery could use Oracle Scheduler or a message queue.
- The performance study uses synthetic, uniformly distributed data on a single instance.

8. Conclusion

8.1 Learning Outcomes

Through this individual project I designed a relational schema from requirements, mapped an ER model to relations and justified each step of normalisation up to BCNF. I implemented DDL and DML, constraints, single-row and group functions, subqueries, joins and views, and used PL/SQL functions, procedures, packages, triggers and cursors for real application behaviour. I learned why application-level checks fail under concurrency, how Oracle's read consistency and row locks interact, how to avoid mutating-table errors and deadlocks, and why transaction boundaries belong to the caller. I also learned to measure rather than assume the effect of indexes by reading execution plans. These outcomes correspond to the course outcomes of designing the structure and operation of the relational data model and examining real-world data requirements to design a database system.

8.2 Future Scope

Recurring and multi-day reservations; a waiting list that promotes the next request when a booking is cancelled; e-mail and calendar (iCalendar) integration; scheduled jobs with `DBMS_SCHEDULER` for expiry, escalation and snapshots; QR-code check-in to measure real occupancy rather than booked time; partitioning BOOKINGS by date for multi-year history; and Oracle Virtual Private Database policies so that row-level access rules are enforced in the database as well as in the API.

Bibliography

[1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.

[2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[3] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. New York, NY, USA: McGraw-Hill, 2003.

[4] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970, doi: 10.1145/362384.362685.

[5] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger, "The notions of consistency and predicate locks in a database system," *Commun. ACM*, vol. 19, no. 11, pp. 624–633, Nov. 1976, doi: 10.1145/360363.360369.

[6] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, San Jose, CA, USA, 1995, pp. 1–10, doi: 10.1145/223784.223785.

[7] J. F. Allen, "Maintaining knowledge about temporal intervals," *Commun. ACM*, vol. 26, no. 11, pp. 832–843, Nov. 1983, doi: 10.1145/182.358434.

[8] Oracle Corporation, *Oracle Database PL/SQL Language Reference, 19c*. [Online]. Available: `https://docs.oracle.com/en/database/oracle/oracle-database/19/lnpls/`

[9] Oracle Corporation, "Manual data locking," in *Oracle Database SQL Language Reference, 19c*. [Online]. Available: `https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/Manual-Data-Locking.html`

[10] Oracle Corporation, *Oracle Database Concepts, 19c* (chapter "Data Concurrency and Consistency"). [Online]. Available: `https://docs.oracle.com/en/database/oracle/oracle-database/19/cncpt/`

[11] Oracle Corporation, *node-oracledb Documentation*. [Online]. Available: `https://node-oracledb.readthedocs.io/`

[12] Oracle Corporation, "Connect Node.js and other scripting languages without a wallet," *Oracle Autonomous Database Documentation*. [Online]. Available: `https://docs.oracle.com/en-us/iaas/autonomous-database-shared/doc/connecting-nodejs-tls.html`

[13] Render, “Blueprint YAML reference.” [Online]. Available: https://render.com/docs/yaml-spec

A. Installation and Execution

```
# 1. Database: create user campus (see DEPLOYMENT_GUIDE.md), then
cd database && sql campus/<password>@localhost:1521/FREEPDB1 @run_all.sql
# 2. API
cd ../backend && cp ../.env.example .env && npm install && npm run dev
# 3. Front end
cd ../frontend && npm install && npm run dev        # http://localhost:5173
# Demo accounts (synthetic): rohan.g@ / kiran.menon@ / coord.scope@ / admin@campusreserve
    .example
# Password for all demo accounts: Campus@2026
```

Listing A.1: Local installation

B. Application Error Codes

Table B.1: Application error codes raised by PL/SQL and their HTTP mapping

Code	Meaning	HTTP
-20001	invalid interval	400
-20002	resource not found	404
-20003	resource under maintenance	409
-20004	resource retired	409
-20005	booking conflict	409
-20006	capacity exceeded	400
-20007	blackout period	409
-20008	outside operating hours	400
-20009	exceeds maximum duration	400
-20010	booking not found	404
-20011	invalid status transition	409
-20012	not authorised	403
-20013	user not found or inactive	403
-20014	maintenance request not found	404
-20015	start time in the past	400
-20016	invalid argument	400
-20017	overlap detected by trigger	409
-20018	resource busy (lock wait timeout)	409
-20019	insufficient notice	400

C. Additional Diagrams and Evidence

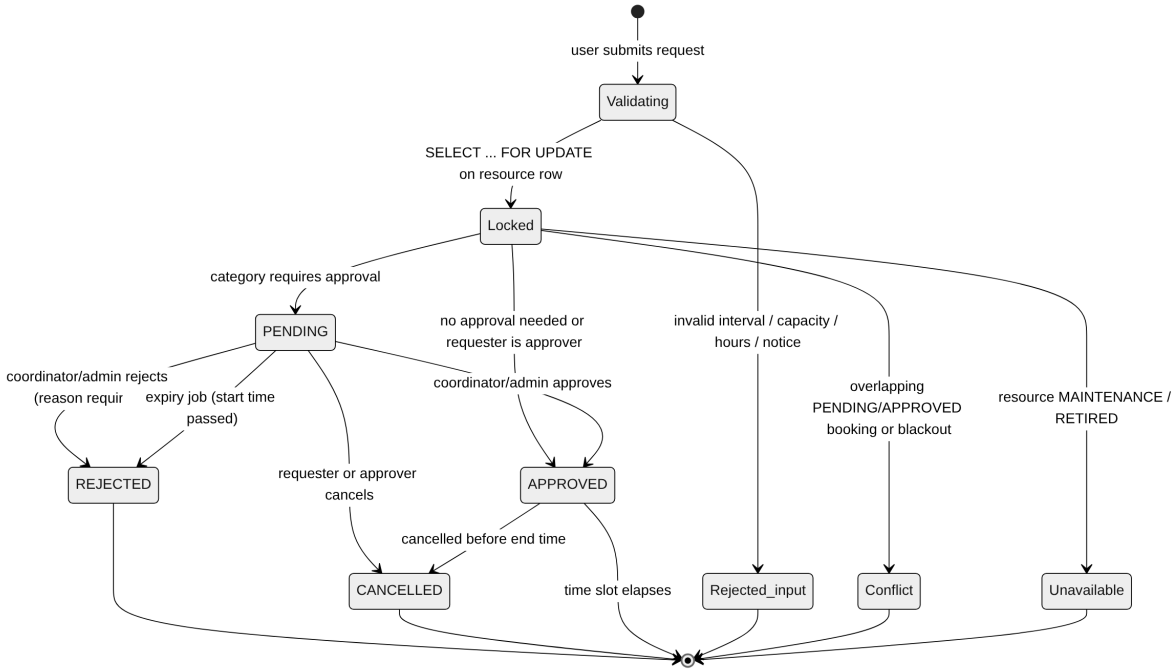


Figure C.1: Booking life cycle enforced by sp_create_booking and trg_bookings_biu

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig07_run_all.png` and rebuild.

Figure C.2: run_all.sql completing with "All objects VALID"

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig08_objects_valid.png` and rebuild.

Figure C.3: Verification V1–V2: every object VALID, no compilation errors

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig08_constraints.png` and rebuild.

Figure C.4: Verification V4–V5: constraints per table; no unindexed foreign keys

EVIDENCE PLACEHOLDER — NOT YET CAPTURED

Replace with a screenshot from your own execution.

Save as `report/figures/evidence/fig10_two_sessions_naive.png` and rebuild.

Figure C.5: Two-session naive check-then-insert and the resulting overlap query

D. Source Code

The complete source code (all SQL scripts, the API and the front end) is in the submission archive and the project repository. The stored procedures and triggers — the core of the concurrency and integrity design — are reproduced in full below; the functions and package specifications are shown in Chapter 4.

D.1 Stored procedures (06_procedures.sql)

```
-------------------------------------------------------------------------------
-- CampusReserve | 06_procedures.sql
-- PURPOSE : Stand-alone stored procedures - the ONLY write path the API uses
--           for bookings and maintenance.
--
-- TRANSACTION CONTRACT
--   None of these procedures COMMIT or ROLLBACK. The caller (the Node.js API,
--   or a SQL*Plus test script) owns the transaction and decides when to commit.
--   This keeps a multi-step API operation atomic and lets tests roll back.
--
-- CONCURRENCY CONTRACT
--   Every write that depends on the current state of a resource first locks
--   that resource row with SELECT ... FOR UPDATE WAIT 5. All writers therefore
--   queue on the same row, and the overlap check that follows sees every
--   committed competing booking. Lock order is always RESOURCE -> BOOKING /
--   MAINTENANCE_REQUEST to avoid deadlocks.
--
-- APPLICATION ERROR CODES (mapped to HTTP status codes by the API)
--   -20001 invalid interval          -20011 invalid status transition
--   -20002 resource not found        -20012 not authorised
--   -20003 resource in maintenance   -20013 user not found / inactive
--   -20004 resource retired          -20014 maintenance request not found
--   -20005 booking conflict          -20015 start time in the past
--   -20006 capacity exceeded         -20016 invalid argument
--   -20007 blackout period           -20017 overlap caught by trigger
--   -20008 outside operating hours   -20018 resource busy (lock timeout)
--   -20009 exceeds maximum duration  -20019 insufficient notice
--   -20010 booking not found
-------------------------------------------------------------------------------
SET DEFINE OFF

-- 1. CREATE BOOKING ----------------------------------------------------------
-- IN : p_resource_id, p_user_id, p_title, p_purpose, p_attendees, p_start, p_end,
--      p_request_key (optional idempotency key supplied by the client)
-- OUT: o_booking_id, o_status (PENDING | APPROVED), o_duplicate (1 = an earlier
--      request with the same key already exists and is returned unchanged)
CREATE OR REPLACE PROCEDURE sp_create_booking (
    p_resource_id  IN  bookings.resource_id%TYPE,
    p_user_id      IN  bookings.requested_by%TYPE,
    p_title        IN  bookings.title%TYPE,
    p_purpose      IN  bookings.purpose%TYPE,
    p_attendees    IN  bookings.attendees%TYPE,
    p_start        IN  bookings.start_time%TYPE,
    p_end          IN  bookings.end_time%TYPE,
    p_request_key  IN  bookings.request_key%TYPE,
    o_booking_id   OUT bookings.booking_id%TYPE,
    o_status       OUT bookings.status%TYPE,
    o_duplicate    OUT NUMBER
)
IS
    e_lock_timeout EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_lock_timeout, -30006);

    v_role_code     roles.role_code%TYPE;
```

```
    v_can_approve   roles.can_approve%TYPE;
    v_res_status    resources.status%TYPE;
    v_capacity      resources.capacity%TYPE;
    v_open          resources.opens_at_hour%TYPE;
    v_close         resources.closes_at_hour%TYPE;
    v_res_name      resources.resource_name%TYPE;
    v_res_dept      resources.managed_by_dept%TYPE;
    v_requires_appr resource_categories.requires_approval%TYPE;
    v_max_hours     resource_categories.max_booking_hours%TYPE;
    v_notice_hours  resource_categories.min_notice_hours%TYPE;
    v_conflict_id   bookings.booking_id%TYPE;
    v_day_start     TIMESTAMP;
    v_count         PLS_INTEGER;
BEGIN
    o_duplicate := 0;

    -- (a) Requester must exist and be active.
    BEGIN
        SELECT ro.role_code, ro.can_approve
          INTO v_role_code, v_can_approve
          FROM users u JOIN roles ro ON ro.role_id = u.role_id
         WHERE u.user_id = p_user_id AND u.is_active = 'Y';
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20013, 'User ' || p_user_id || ' not found or inactive.');
    END;

    -- (b) Idempotency: a retried request returns the original booking.
    IF p_request_key IS NOT NULL THEN
        BEGIN
            SELECT booking_id, status INTO o_booking_id, o_status
              FROM bookings
             WHERE request_key = p_request_key AND requested_by = p_user_id;
            o_duplicate := 1;
            RETURN;
        EXCEPTION
            WHEN NO_DATA_FOUND THEN NULL;
        END;
    END IF;

    -- (c) Interval sanity (cheap checks before taking any lock).
    IF p_start IS NULL OR p_end IS NULL OR p_end <= p_start THEN
        RAISE_APPLICATION_ERROR(-20001, 'End time must be later than start time.');
    END IF;
    IF p_start <= SYSTIMESTAMP THEN
        RAISE_APPLICATION_ERROR(-20015, 'Bookings must start in the future.');
    END IF;
    IF p_attendees IS NULL OR p_attendees < 1 THEN
        RAISE_APPLICATION_ERROR(-20016, 'Attendees must be at least 1.');
    END IF;

    -- (d) SERIALISATION POINT: lock the resource row. A concurrent caller for the
    --     same resource waits here (up to 5 s) until this transaction ends.
    BEGIN
        SELECT r.status, r.capacity, r.opens_at_hour, r.closes_at_hour, r.resource_name,
               r.managed_by_dept, c.requires_approval, c.max_booking_hours, c.min_notice_hours
          INTO v_res_status, v_capacity, v_open, v_close, v_res_name,
               v_res_dept, v_requires_appr, v_max_hours, v_notice_hours
          FROM resources r
          JOIN resource_categories c ON c.category_id = r.category_id
         WHERE r.resource_id = p_resource_id
           FOR UPDATE OF r.status WAIT 5;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20002, 'Resource ' || p_resource_id || ' does not exist.');
        WHEN e_lock_timeout THEN
            RAISE_APPLICATION_ERROR(-20018, 'Resource is busy with another booking; please retry.');
    END;

    -- (e) Resource-level rules (evaluated while holding the lock).
    IF v_res_status = 'MAINTENANCE' THEN
        RAISE_APPLICATION_ERROR(-20003, v_res_name || ' is under maintenance and cannot be booked.');
    ELSIF v_res_status = 'RETIRED' THEN
```

```
        RAISE_APPLICATION_ERROR(-20004, v_res_name || ' has been retired.');
    END IF;

    IF v_capacity > 0 AND p_attendees > v_capacity THEN
        RAISE_APPLICATION_ERROR(-20006, 'Attendees (' || p_attendees || ') exceed capacity ('
                                        || v_capacity || ').');
    END IF;

    v_day_start := CAST(TRUNC(CAST(p_start AS DATE)) AS TIMESTAMP);
    IF p_start < v_day_start + NUMTODSINTERVAL(v_open, 'HOUR')
       OR p_end > v_day_start + NUMTODSINTERVAL(v_close, 'HOUR') THEN
        RAISE_APPLICATION_ERROR(-20008, 'Bookings must fall within operating hours '
                                || LPAD(v_open, 2, '0') || ':00-' || LPAD(v_close, 2, '0') || ':00 on one day.
     ');
    END IF;

    IF fn_hours_between(p_start, p_end) > v_max_hours THEN
        RAISE_APPLICATION_ERROR(-20009, 'Maximum booking length for this category is '
                                        || v_max_hours || ' hours.');
    END IF;

    IF v_can_approve = 'N'
       AND p_start < SYSTIMESTAMP + NUMTODSINTERVAL(v_notice_hours, 'HOUR') THEN
        RAISE_APPLICATION_ERROR(-20019, 'This resource needs at least ' || v_notice_hours
                                        || ' hours notice.');
    END IF;

    SELECT COUNT(*) INTO v_count
      FROM resource_blackouts
     WHERE resource_id = p_resource_id
       AND start_time < p_end AND end_time > p_start;
    IF v_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20007, 'The resource is blocked out during the requested time.');
    END IF;

    -- (f) Overlap check. [s1,e1) overlaps [s2,e2) iff s1 < e2 AND s2 < e1.
    --     PENDING requests also hold the slot so two users cannot both queue for it.
    SELECT MIN(booking_id) INTO v_conflict_id
      FROM bookings
     WHERE resource_id = p_resource_id
       AND status IN ('PENDING', 'APPROVED')
       AND start_time < p_end
       AND end_time   > p_start;
    IF v_conflict_id IS NOT NULL THEN
        RAISE_APPLICATION_ERROR(-20005, 'Time slot conflicts with booking #' || v_conflict_id || '.');
    END IF;

    -- (g) Accept. Approvers and categories without approval are auto-approved.
    o_status := CASE WHEN v_requires_appr = 'N' OR v_can_approve = 'Y'
                     THEN 'APPROVED' ELSE 'PENDING' END;

    INSERT INTO bookings (resource_id, requested_by, title, purpose, attendees,
                          start_time, end_time, status, request_key)
    VALUES (p_resource_id, p_user_id, TRIM(p_title), p_purpose, p_attendees,
            p_start, p_end, o_status, p_request_key)
    RETURNING booking_id INTO o_booking_id;

    IF v_requires_appr = 'Y' AND o_status = 'APPROVED' THEN
        INSERT INTO booking_approvals (booking_id, approver_id, decision, remarks)
        VALUES (o_booking_id, p_user_id, 'APPROVED', 'Auto-approved: requester holds approver role');
    END IF;

    INSERT INTO notifications (user_id, notif_type, title, message, booking_id)
    VALUES (p_user_id, 'BOOKING',
            CASE o_status WHEN 'APPROVED' THEN 'Booking confirmed' ELSE 'Booking submitted' END,
            v_res_name || ': ' || TO_CHAR(p_start, 'DD-Mon HH24:MI') || '-' || TO_CHAR(p_end, 'HH24:MI')
            || CASE o_status WHEN 'PENDING' THEN ' is awaiting approval.' ELSE ' is confirmed.' END,
            o_booking_id);

    IF o_status = 'PENDING' THEN
        -- Notify every approver who may decide this booking.
        INSERT INTO notifications (user_id, notif_type, title, message, booking_id)
        SELECT u.user_id, 'APPROVAL', 'Approval required',
```

```
               'New request #' || o_booking_id || ' for ' || v_res_name || '.', o_booking_id
          FROM users u JOIN roles ro ON ro.role_id = u.role_id
         WHERE u.is_active = 'Y'
           AND (ro.role_code = 'ADMIN'
                OR (ro.role_code = 'COORDINATOR' AND u.dept_id = v_res_dept));
    END IF;
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        -- Two retries with the same key raced past step (b): return the winner.
        IF p_request_key IS NULL THEN
            RAISE;
        END IF;
        BEGIN
            SELECT booking_id, status INTO o_booking_id, o_status
              FROM bookings
             WHERE request_key = p_request_key AND requested_by = p_user_id;
            o_duplicate := 1;
        EXCEPTION
            WHEN NO_DATA_FOUND THEN
                RAISE_APPLICATION_ERROR(-20016, 'This request key is already in use.');
        END;
END sp_create_booking;
/

-- 2. APPROVE / REJECT BOOKING ------------------------------------------------
-- IN : p_booking_id, p_approver_id, p_decision ('APPROVED' | 'REJECTED'), p_remarks
-- Authorisation: ADMIN may decide any booking; COORDINATOR only bookings of
-- resources managed by the coordinator's own department.
CREATE OR REPLACE PROCEDURE sp_decide_booking (
    p_booking_id  IN bookings.booking_id%TYPE,
    p_approver_id IN users.user_id%TYPE,
    p_decision    IN booking_approvals.decision%TYPE,
    p_remarks     IN booking_approvals.remarks%TYPE
)
IS
    e_lock_timeout EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_lock_timeout, -30006);
    v_role_code   roles.role_code%TYPE;
    v_appr_dept   users.dept_id%TYPE;
    v_resource_id bookings.resource_id%TYPE;
    v_res_dept    resources.managed_by_dept%TYPE;
    v_res_status  resources.status%TYPE;
    v_res_name    resources.resource_name%TYPE;
    v_status      bookings.status%TYPE;
    v_start       bookings.start_time%TYPE;
    v_end         bookings.end_time%TYPE;
    v_requester   bookings.requested_by%TYPE;
    v_conflict_id bookings.booking_id%TYPE;
BEGIN
    IF p_decision NOT IN ('APPROVED', 'REJECTED') OR p_decision IS NULL THEN
        RAISE_APPLICATION_ERROR(-20016, 'Decision must be APPROVED or REJECTED.');
    END IF;
    IF p_decision = 'REJECTED' AND TRIM(p_remarks) IS NULL THEN
        RAISE_APPLICATION_ERROR(-20016, 'A reason is required when rejecting a booking.');
    END IF;

    BEGIN
        SELECT ro.role_code, u.dept_id INTO v_role_code, v_appr_dept
          FROM users u JOIN roles ro ON ro.role_id = u.role_id
         WHERE u.user_id = p_approver_id AND u.is_active = 'Y' AND ro.can_approve = 'Y';
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20012, 'User ' || p_approver_id || ' is not allowed to approve bookings.')
     ;
    END;

    -- Find the resource without locking, then lock in the global order RESOURCE -> BOOKING.
    BEGIN
        SELECT resource_id INTO v_resource_id FROM bookings WHERE booking_id = p_booking_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20010, 'Booking ' || p_booking_id || ' does not exist.');
    END;
```

```
    BEGIN
        SELECT managed_by_dept, status, resource_name INTO v_res_dept, v_res_status, v_res_name
          FROM resources WHERE resource_id = v_resource_id
           FOR UPDATE WAIT 5;
        SELECT status, start_time, end_time, requested_by
          INTO v_status, v_start, v_end, v_requester
          FROM bookings WHERE booking_id = p_booking_id
           FOR UPDATE WAIT 5;
    EXCEPTION
        WHEN e_lock_timeout THEN
            RAISE_APPLICATION_ERROR(-20018, 'Booking is being modified by another user; please retry.');
    END;

    IF v_role_code = 'COORDINATOR' AND v_appr_dept <> v_res_dept THEN
        RAISE_APPLICATION_ERROR(-20012, 'Coordinators may only decide bookings for their own department''s
     resources.');
    END IF;
    IF v_status <> 'PENDING' THEN
        RAISE_APPLICATION_ERROR(-20011, 'Only PENDING bookings can be decided (current: ' || v_status || ').');
    END IF;

    IF p_decision = 'APPROVED' THEN
        IF v_start <= SYSTIMESTAMP THEN
            RAISE_APPLICATION_ERROR(-20015, 'Cannot approve a booking whose start time has passed.');
        END IF;
        IF v_res_status <> 'AVAILABLE' THEN
            RAISE_APPLICATION_ERROR(-20003, v_res_name || ' is not available (' || v_res_status || ').');
        END IF;
        -- Defensive re-check: no other APPROVED booking may overlap.
        SELECT MIN(booking_id) INTO v_conflict_id
          FROM bookings
         WHERE resource_id = v_resource_id AND status = 'APPROVED'
           AND booking_id <> p_booking_id
           AND start_time < v_end AND end_time > v_start;
        IF v_conflict_id IS NOT NULL THEN
            RAISE_APPLICATION_ERROR(-20005, 'Conflicts with approved booking #' || v_conflict_id || '.');
        END IF;
    END IF;

    UPDATE bookings SET status = p_decision WHERE booking_id = p_booking_id;

    INSERT INTO booking_approvals (booking_id, approver_id, decision, remarks)
    VALUES (p_booking_id, p_approver_id, p_decision, p_remarks);

    INSERT INTO notifications (user_id, notif_type, title, message, booking_id)
    VALUES (v_requester, 'APPROVAL',
            'Booking ' || LOWER(p_decision),
            v_res_name || ' on ' || TO_CHAR(v_start, 'DD-Mon HH24:MI') || ' was ' || LOWER(p_decision)
            || CASE WHEN p_remarks IS NOT NULL THEN ': ' || SUBSTR(p_remarks, 1, 200) END,
            p_booking_id);
END sp_decide_booking;
/

-- 3. CANCEL BOOKING ---------------------------------------------------------
-- The requester may cancel their own booking; approvers/admins may cancel any.
-- Cancelling frees the slot immediately because the overlap check only
-- considers PENDING and APPROVED rows.
CREATE OR REPLACE PROCEDURE sp_cancel_booking (
    p_booking_id IN bookings.booking_id%TYPE,
    p_user_id    IN users.user_id%TYPE,
    p_reason     IN bookings.cancel_reason%TYPE
)
IS
    e_lock_timeout EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_lock_timeout, -30006);
    v_can_approve roles.can_approve%TYPE;
    v_requester   bookings.requested_by%TYPE;
    v_status      bookings.status%TYPE;
    v_end         bookings.end_time%TYPE;
    v_start       bookings.start_time%TYPE;
    v_resource_id bookings.resource_id%TYPE;
BEGIN
```

```
    BEGIN
        SELECT ro.can_approve INTO v_can_approve
          FROM users u JOIN roles ro ON ro.role_id = u.role_id
         WHERE u.user_id = p_user_id AND u.is_active = 'Y';
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20013, 'User ' || p_user_id || ' not found or inactive.');
    END;

    BEGIN
        SELECT resource_id INTO v_resource_id FROM bookings WHERE booking_id = p_booking_id;
        -- Same lock order as booking creation: resource first.
        SELECT resource_id INTO v_resource_id FROM resources
         WHERE resource_id = v_resource_id FOR UPDATE WAIT 5;
        SELECT requested_by, status, start_time, end_time
          INTO v_requester, v_status, v_start, v_end
          FROM bookings WHERE booking_id = p_booking_id FOR UPDATE WAIT 5;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20010, 'Booking ' || p_booking_id || ' does not exist.');
        WHEN e_lock_timeout THEN
            RAISE_APPLICATION_ERROR(-20018, 'Booking is being modified by another user; please retry.');
    END;

    IF v_requester <> p_user_id AND v_can_approve = 'N' THEN
        RAISE_APPLICATION_ERROR(-20012, 'You can only cancel your own bookings.');
    END IF;
    IF v_status NOT IN ('PENDING', 'APPROVED') THEN
        RAISE_APPLICATION_ERROR(-20011, 'Booking is already ' || v_status || '.');
    END IF;
    IF v_end <= SYSTIMESTAMP THEN
        RAISE_APPLICATION_ERROR(-20011, 'A booking that has already ended cannot be cancelled.');
    END IF;

    UPDATE bookings
       SET status = 'CANCELLED',
           cancelled_at = SYSTIMESTAMP,
           cancel_reason = NVL(TRIM(p_reason), 'No reason given')
     WHERE booking_id = p_booking_id;

    IF v_requester <> p_user_id THEN
        INSERT INTO notifications (user_id, notif_type, title, message, booking_id)
        VALUES (v_requester, 'BOOKING', 'Booking cancelled by staff',
                'Your booking #' || p_booking_id || ' on ' || TO_CHAR(v_start, 'DD-Mon HH24:MI')
                || ' was cancelled: ' || SUBSTR(NVL(p_reason, 'no reason given'), 1, 200),
                p_booking_id);
    END IF;
END sp_cancel_booking;
/

-- 4. SUBMIT MAINTENANCE REQUEST ------------------------------------------------
-- p_blocks_booking = 'Y' takes the resource out of service immediately; only
-- staff (faculty, coordinators, admins) may do that. Holders of upcoming
-- bookings on the resource are notified through a cursor FOR loop.
CREATE OR REPLACE PROCEDURE sp_submit_maintenance (
    p_resource_id    IN  maintenance_requests.resource_id%TYPE,
    p_user_id        IN  maintenance_requests.reported_by%TYPE,
    p_title          IN  maintenance_requests.title%TYPE,
    p_description    IN  maintenance_requests.description%TYPE,
    p_priority       IN  maintenance_requests.priority%TYPE,
    p_blocks_booking IN  maintenance_requests.blocks_booking%TYPE,
    o_request_id     OUT maintenance_requests.request_id%TYPE
)
IS
    v_role_code roles.role_code%TYPE;
    v_res_name  resources.resource_name%TYPE;
    v_res_dept  resources.managed_by_dept%TYPE;
    v_blocks    CHAR(1) := NVL(UPPER(p_blocks_booking), 'N');
BEGIN
    IF NVL(UPPER(p_priority), 'X') NOT IN ('LOW','MEDIUM','HIGH','CRITICAL') THEN
        RAISE_APPLICATION_ERROR(-20016, 'Priority must be LOW, MEDIUM, HIGH or CRITICAL.');
    END IF;
    IF v_blocks NOT IN ('Y', 'N') THEN
```

```
        RAISE_APPLICATION_ERROR(-20016, 'blocks_booking must be Y or N.');
    END IF;

    BEGIN
        SELECT ro.role_code INTO v_role_code
          FROM users u JOIN roles ro ON ro.role_id = u.role_id
         WHERE u.user_id = p_user_id AND u.is_active = 'Y';
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20013, 'User ' || p_user_id || ' not found or inactive.');
    END;
    IF v_blocks = 'Y' AND v_role_code = 'STUDENT' THEN
        RAISE_APPLICATION_ERROR(-20012, 'Only staff can take a resource out of service.');
    END IF;

    BEGIN
        SELECT resource_name, managed_by_dept INTO v_res_name, v_res_dept
          FROM resources WHERE resource_id = p_resource_id
           FOR UPDATE WAIT 5;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20002, 'Resource ' || p_resource_id || ' does not exist.');
    END;

    INSERT INTO maintenance_requests (resource_id, reported_by, title, description,
                                      priority, status, blocks_booking)
    VALUES (p_resource_id, p_user_id, TRIM(p_title), p_description,
            UPPER(p_priority), 'OPEN', v_blocks)
    RETURNING request_id INTO o_request_id;

    IF v_blocks = 'Y' THEN
        UPDATE resources SET status = 'MAINTENANCE', updated_at = SYSTIMESTAMP
         WHERE resource_id = p_resource_id AND status = 'AVAILABLE';

        -- Cursor FOR loop: warn everyone holding an upcoming booking.
        FOR b IN (SELECT booking_id, requested_by, start_time
                    FROM bookings
                   WHERE resource_id = p_resource_id
                     AND status IN ('PENDING', 'APPROVED')
                     AND end_time > SYSTIMESTAMP)
        LOOP
            INSERT INTO notifications (user_id, notif_type, title, message, booking_id)
            VALUES (b.requested_by, 'MAINTENANCE', 'Resource under maintenance',
                    v_res_name || ' was taken out of service. Your booking on '
                    || TO_CHAR(b.start_time, 'DD-Mon HH24:MI') || ' may be affected.',
                    b.booking_id);
        END LOOP;
    END IF;

    -- Alert the staff responsible for the resource.
    INSERT INTO notifications (user_id, notif_type, title, message)
    SELECT u.user_id, 'MAINTENANCE', 'New maintenance request',
           '[' || UPPER(p_priority) || '] ' || v_res_name || ': ' || SUBSTR(p_title, 1, 100)
      FROM users u JOIN roles ro ON ro.role_id = u.role_id
     WHERE u.is_active = 'Y'
       AND (ro.role_code = 'ADMIN' OR (ro.role_code = 'COORDINATOR' AND u.dept_id = v_res_dept));
END sp_submit_maintenance;
/

-- 5. UPDATE MAINTENANCE STATUS -----------------------------------------------------
-- Moves a ticket through OPEN -> IN_PROGRESS -> RESOLVED -> CLOSED (transition
-- rules are enforced by TRG_MAINTENANCE_BIU) and then recomputes the resource
-- status: MAINTENANCE while any blocking ticket is open, otherwise AVAILABLE.
CREATE OR REPLACE PROCEDURE sp_update_maintenance_status (
    p_request_id  IN maintenance_requests.request_id%TYPE,
    p_actor_id    IN users.user_id%TYPE,
    p_new_status  IN maintenance_requests.status%TYPE,
    p_assigned_to IN maintenance_requests.assigned_to%TYPE,
    p_notes       IN maintenance_requests.resolution_notes%TYPE
)
IS
    v_can_approve  roles.can_approve%TYPE;
    v_resource_id  maintenance_requests.resource_id%TYPE;
```

```
    v_reporter     maintenance_requests.reported_by%TYPE;
    v_title        maintenance_requests.title%TYPE;
    v_res_status   resources.status%TYPE;
    v_open_blocks  PLS_INTEGER;
BEGIN
    IF NVL(UPPER(p_new_status), 'X') NOT IN ('OPEN','IN_PROGRESS','RESOLVED','CLOSED') THEN
        RAISE_APPLICATION_ERROR(-20016, 'Unknown maintenance status: ' || p_new_status);
    END IF;

    BEGIN
        SELECT ro.can_approve INTO v_can_approve
          FROM users u JOIN roles ro ON ro.role_id = u.role_id
         WHERE u.user_id = p_actor_id AND u.is_active = 'Y';
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20013, 'User ' || p_actor_id || ' not found or inactive.');
    END;
    IF v_can_approve = 'N' THEN
        RAISE_APPLICATION_ERROR(-20012, 'Only coordinators and administrators manage maintenance.');
    END IF;

    BEGIN
        SELECT resource_id INTO v_resource_id
          FROM maintenance_requests WHERE request_id = p_request_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20014, 'Maintenance request ' || p_request_id || ' does not exist.');
    END;

    -- Lock order: resource, then ticket.
    SELECT status INTO v_res_status FROM resources
     WHERE resource_id = v_resource_id FOR UPDATE WAIT 5;
    SELECT reported_by, title INTO v_reporter, v_title
      FROM maintenance_requests WHERE request_id = p_request_id FOR UPDATE WAIT 5;

    UPDATE maintenance_requests
       SET status           = UPPER(p_new_status),
           assigned_to      = NVL(p_assigned_to, assigned_to),
           resolution_notes = NVL(p_notes, resolution_notes)
     WHERE request_id = p_request_id;

    SELECT COUNT(*) INTO v_open_blocks
      FROM maintenance_requests
     WHERE resource_id = v_resource_id
       AND blocks_booking = 'Y'
       AND status IN ('OPEN', 'IN_PROGRESS');

    IF v_open_blocks > 0 AND v_res_status = 'AVAILABLE' THEN
        UPDATE resources SET status = 'MAINTENANCE', updated_at = SYSTIMESTAMP
         WHERE resource_id = v_resource_id;
    ELSIF v_open_blocks = 0 AND v_res_status = 'MAINTENANCE' THEN
        UPDATE resources SET status = 'AVAILABLE', updated_at = SYSTIMESTAMP
         WHERE resource_id = v_resource_id;
    END IF;

    INSERT INTO notifications (user_id, notif_type, title, message)
    VALUES (v_reporter, 'MAINTENANCE', 'Maintenance update',
            'Your request "' || SUBSTR(v_title, 1, 80) || '" is now ' || UPPER(p_new_status) || '.');
END sp_update_maintenance_status;
/
```

D.2 Triggers (09_triggers.sql)

```
-------------------------------------------------------------------------------
-- CampusReserve | 09_triggers.sql
-- PURPOSE : Row-level validation, a COMPOUND trigger for overlap detection,
--           and audit triggers.
--
```

```
-- MUTATING-TABLE SAFETY
--   A row-level trigger on BOOKINGS may not query BOOKINGS (ORA-04091). The
--   overlap check therefore runs in the AFTER STATEMENT section of a compound
--   trigger, when the table is no longer mutating. Row-level triggers only read
--   OTHER tables (RESOURCES), which is allowed.
--
-- ACTOR ATTRIBUTION
--   The API sets the Oracle CLIENT_IDENTIFIER to the signed-in user's ID on every
--   call (node-oracledb: connection.clientId). Audit triggers read it with
--   SYS_CONTEXT('USERENV','CLIENT_IDENTIFIER') (via FN_CURRENT_ACTOR) and fall back to the row's own
--   user column when it is not set (e.g. SQL*Plus sessions).
--------------------------------------------------------------------------------
SET DEFINE OFF

-- 1. USERS: normalise e-mail so UNIQUE(email) is effectively case-insensitive
CREATE OR REPLACE TRIGGER trg_users_biu
BEFORE INSERT OR UPDATE OF email, full_name ON users
FOR EACH ROW
BEGIN
    :NEW.email     := LOWER(TRIM(:NEW.email));
    :NEW.full_name := TRIM(:NEW.full_name);
END;
/

-- 2. RESOURCES: maintain updated_at
CREATE OR REPLACE TRIGGER trg_resources_bu
BEFORE UPDATE ON resources
FOR EACH ROW
BEGIN
    :NEW.updated_at := SYSTIMESTAMP;
END;
/

-- 3. BOOKINGS: row validation and status state machine
--    INSERT : new rows must be PENDING or APPROVED; resource must be AVAILABLE;
--             attendees must fit the room.
--    UPDATE : PENDING  -> APPROVED | REJECTED | CANCELLED
--             APPROVED -> CANCELLED
--             REJECTED / CANCELLED are final.
CREATE OR REPLACE TRIGGER trg_bookings_biu
BEFORE INSERT OR UPDATE ON bookings
FOR EACH ROW
DECLARE
    v_res_status resources.status%TYPE;
    v_capacity   resources.capacity%TYPE;
BEGIN
    :NEW.updated_at := SYSTIMESTAMP;

    IF INSERTING THEN
        IF :NEW.status NOT IN ('PENDING', 'APPROVED') THEN
            RAISE_APPLICATION_ERROR(-20011, 'New bookings must be PENDING or APPROVED.');
        END IF;
        SELECT status, capacity INTO v_res_status, v_capacity
          FROM resources WHERE resource_id = :NEW.resource_id;
        IF v_res_status = 'MAINTENANCE' THEN
            RAISE_APPLICATION_ERROR(-20003, 'Resource is under maintenance.');
        ELSIF v_res_status = 'RETIRED' THEN
            RAISE_APPLICATION_ERROR(-20004, 'Resource has been retired.');
        END IF;
        IF v_capacity > 0 AND :NEW.attendees > v_capacity THEN
            RAISE_APPLICATION_ERROR(-20006, 'Attendees exceed resource capacity.');
        END IF;
    END IF;

    IF UPDATING THEN
        IF :OLD.status <> :NEW.status THEN
            IF NOT (   (:OLD.status = 'PENDING'  AND :NEW.status IN ('APPROVED','REJECTED','CANCELLED'))
                    OR (:OLD.status = 'APPROVED' AND :NEW.status = 'CANCELLED')) THEN
                RAISE_APPLICATION_ERROR(-20011, 'Illegal booking status change '
                                        || :OLD.status || ' -> ' || :NEW.status || '.');
            END IF;
        END IF;
        IF :NEW.status = 'CANCELLED' AND :NEW.cancelled_at IS NULL THEN
```

```
            :NEW.cancelled_at := SYSTIMESTAMP;
        END IF;
        -- Time / resource can only be edited while the booking is still live.
        IF (   :OLD.start_time  <> :NEW.start_time
            OR :OLD.end_time    <> :NEW.end_time
            OR :OLD.resource_id <> :NEW.resource_id)
           AND :OLD.status NOT IN ('PENDING', 'APPROVED') THEN
            RAISE_APPLICATION_ERROR(-20011, 'A ' || :OLD.status || ' booking cannot be rescheduled.');
        END IF;
    END IF;
END;
/

-- 4. BOOKINGS: defence-in-depth overlap detection (COMPOUND trigger)
--    Catches overlaps created by ANY writer (ad-hoc SQL, a buggy script), not
--    just SP_CREATE_BOOKING. It cannot by itself stop two *concurrent*
--    transactions (each sees only its own uncommitted row) - that is what the
--    resource row lock in SP_CREATE_BOOKING is for. See 12_transaction_tests.sql.
CREATE OR REPLACE TRIGGER trg_bookings_no_overlap
FOR INSERT OR UPDATE OF start_time, end_time, status, resource_id ON bookings
COMPOUND TRIGGER
    TYPE t_ids IS TABLE OF bookings.booking_id%TYPE;
    g_ids t_ids := t_ids();

    AFTER EACH ROW IS
    BEGIN
        IF :NEW.status IN ('PENDING', 'APPROVED') THEN
            g_ids.EXTEND;
            g_ids(g_ids.LAST) := :NEW.booking_id;
        END IF;
    END AFTER EACH ROW;

    AFTER STATEMENT IS
        v_conflict bookings.booking_id%TYPE;
    BEGIN
        FOR i IN 1 .. g_ids.COUNT LOOP
            SELECT MIN(o.booking_id) INTO v_conflict
              FROM bookings b
              JOIN bookings o
                ON o.resource_id = b.resource_id
               AND o.booking_id <> b.booking_id
               AND o.status IN ('PENDING', 'APPROVED')
               AND o.start_time < b.end_time
               AND o.end_time   > b.start_time
             WHERE b.booking_id = g_ids(i);
            IF v_conflict IS NOT NULL THEN
                RAISE_APPLICATION_ERROR(-20017, 'Booking #' || g_ids(i)
                                        || ' overlaps booking #' || v_conflict || '.');
            END IF;
        END LOOP;
        g_ids.DELETE;
    END AFTER STATEMENT;
END trg_bookings_no_overlap;
/

-- 5. BOOKINGS: audit trail
CREATE OR REPLACE TRIGGER trg_bookings_audit
AFTER INSERT OR UPDATE OR DELETE ON bookings
FOR EACH ROW
DECLARE
    v_actor   NUMBER := fn_current_actor;
    v_details VARCHAR2(1000);
BEGIN
    IF INSERTING THEN
        INSERT INTO audit_logs (table_name, record_id, action, new_status, changed_by, details)
        VALUES ('BOOKINGS', :NEW.booking_id, 'INSERT', :NEW.status, NVL(v_actor, :NEW.requested_by),
                'resource=' || :NEW.resource_id
                || '; slot=' || TO_CHAR(:NEW.start_time, 'YYYY-MM-DD HH24:MI')
                || '..' || TO_CHAR(:NEW.end_time, 'HH24:MI')
                || '; attendees=' || :NEW.attendees);
    ELSIF UPDATING THEN
        IF :OLD.status <> :NEW.status THEN
            v_details := 'status changed';
```

```
        END IF;
        IF :OLD.start_time <> :NEW.start_time OR :OLD.end_time <> :NEW.end_time THEN
            v_details := v_details || CASE WHEN v_details IS NOT NULL THEN '; ' END
                      || 'rescheduled from ' || TO_CHAR(:OLD.start_time, 'YYYY-MM-DD HH24:MI');
        END IF;
        IF :NEW.status = 'CANCELLED' AND :OLD.status <> 'CANCELLED' THEN
            v_details := v_details || '; reason=' || SUBSTR(:NEW.cancel_reason, 1, 300);
        END IF;
        INSERT INTO audit_logs (table_name, record_id, action, old_status, new_status, changed_by, details)
        VALUES ('BOOKINGS', :NEW.booking_id, 'UPDATE', :OLD.status, :NEW.status,
                NVL(v_actor, :NEW.requested_by), NVL(v_details, 'fields updated'));
    ELSE
        INSERT INTO audit_logs (table_name, record_id, action, old_status, changed_by, details)
        VALUES ('BOOKINGS', :OLD.booking_id, 'DELETE', :OLD.status, v_actor,
                'row deleted; resource=' || :OLD.resource_id);
    END IF;
END;
/

-- 6. MAINTENANCE_REQUESTS: validation and ticket state machine
--    OPEN -> IN_PROGRESS | RESOLVED | CLOSED
--    IN_PROGRESS -> OPEN | RESOLVED | CLOSED
--    RESOLVED -> CLOSED | OPEN (re-open)
--    CLOSED is final.
CREATE OR REPLACE TRIGGER trg_maintenance_biu
BEFORE INSERT OR UPDATE ON maintenance_requests
FOR EACH ROW
BEGIN
    :NEW.updated_at := SYSTIMESTAMP;

    IF INSERTING AND :NEW.status <> 'OPEN' THEN
        RAISE_APPLICATION_ERROR(-20011, 'New maintenance requests must start as OPEN.');
    END IF;

    IF UPDATING AND :OLD.status <> :NEW.status THEN
        IF NOT (   (:OLD.status = 'OPEN'        AND :NEW.status IN ('IN_PROGRESS','RESOLVED','CLOSED'))
                OR (:OLD.status = 'IN_PROGRESS' AND :NEW.status IN ('OPEN','RESOLVED','CLOSED'))
                OR (:OLD.status = 'RESOLVED'    AND :NEW.status IN ('CLOSED','OPEN'))) THEN
            RAISE_APPLICATION_ERROR(-20011, 'Illegal maintenance status change '
                                    || :OLD.status || ' -> ' || :NEW.status || '.');
        END IF;
    END IF;

    -- Keep resolved_at consistent with status (also enforced by ck_maint_resolved).
    IF :NEW.status IN ('RESOLVED', 'CLOSED') AND :NEW.resolved_at IS NULL THEN
        :NEW.resolved_at := SYSTIMESTAMP;
    ELSIF :NEW.status IN ('OPEN', 'IN_PROGRESS') THEN
        :NEW.resolved_at := NULL;
    END IF;
END;
/

-- 7. MAINTENANCE_REQUESTS: audit trail
CREATE OR REPLACE TRIGGER trg_maintenance_audit
AFTER INSERT OR UPDATE ON maintenance_requests
FOR EACH ROW
DECLARE
    v_actor   NUMBER := fn_current_actor;
    v_details VARCHAR2(1000);
BEGIN
    IF INSERTING THEN
        INSERT INTO audit_logs (table_name, record_id, action, new_status, changed_by, details)
        VALUES ('MAINTENANCE_REQUESTS', :NEW.request_id, 'INSERT', :NEW.status,
                NVL(v_actor, :NEW.reported_by),
                'resource=' || :NEW.resource_id || '; priority=' || :NEW.priority
                || '; blocks_booking=' || :NEW.blocks_booking);
    ELSE
        IF :OLD.priority <> :NEW.priority THEN
            v_details := 'priority ' || :OLD.priority || ' -> ' || :NEW.priority;
        END IF;
        IF NVL(:OLD.assigned_to, -1) <> NVL(:NEW.assigned_to, -1) THEN
            v_details := v_details || CASE WHEN v_details IS NOT NULL THEN '; ' END
                      || 'assigned_to ' || NVL(TO_CHAR(:OLD.assigned_to), 'none')
```

```
                    || ' -> ' || NVL(TO_CHAR(:NEW.assigned_to), 'none');
        END IF;
        INSERT INTO audit_logs (table_name, record_id, action, old_status, new_status, changed_by, details)
        VALUES ('MAINTENANCE_REQUESTS', :NEW.request_id, 'UPDATE', :OLD.status, :NEW.status,
                v_actor, NVL(v_details, CASE WHEN :OLD.status <> :NEW.status
                                             THEN 'status changed' ELSE 'fields updated' END));
    END IF;
END;
/

-- 8. RESOURCES: audit availability changes
CREATE OR REPLACE TRIGGER trg_resources_status_audit
AFTER UPDATE OF status ON resources
FOR EACH ROW
WHEN (OLD.status <> NEW.status)
BEGIN
    INSERT INTO audit_logs (table_name, record_id, action, old_status, new_status, changed_by, details)
    VALUES ('RESOURCES', :NEW.resource_id, 'UPDATE', :OLD.status, :NEW.status,
            fn_current_actor,
            'resource availability changed');
END;
/

-- 9. AUDIT_LOGS: append-only
CREATE OR REPLACE TRIGGER trg_audit_logs_immutable
BEFORE UPDATE OR DELETE ON audit_logs
BEGIN
    RAISE_APPLICATION_ERROR(-20012, 'AUDIT_LOGS is append-only.');
END;
/
```